

MP IFoE architecture

Part Number: XN-202350-PS

Draft 1K

2025-07-29



Copyright © This controlled document is the property of AMD, Inc.
Any duplication, reproduction, or transmission to unauthorized parties without the express written
permission of AMD Inc. is prohibited.

Copyright © 2025 AMD, Inc. All rights reserved

Table of Contents

1.1	CHANGE LOG	9
1.2	REFERENCES & RELATED DOCUMENTS	9
1.3	GLOSSARY	9
2	INTRODUCTION.....	11
3	SYSTEM OVERVIEW.....	12
3.1	COMMON COMPONENTS.....	15
3.1.1	SMN.....	15
3.1.2	RSMU.....	15
3.2	IFOE SUBSYSTEM	15
3.2.1	XRMAC.....	15
3.2.2	XRPFC	16
3.2.3	XRSEC.....	16
3.2.4	EX.....	16
3.2.5	IFOE	17
3.3	PHY	17
3.3.1	UPI2 BRIDGE	17
3.3.2	GTA	17
3.3.3	I2C (NOT USED)	17
3.4	MP_IFOE	17
3.4.1	CPU	17
3.4.2	SRAM.....	17
3.4.3	MBOX	17
3.4.4	ADMA	17
3.4.5	SMN INTERRUPTS.....	18
3.5	HOST SIDE.....	18
3.5.1	NBIF	18
3.5.2	IEP.....	18
4	REQUIREMENTS ARISING FROM SYSTEM DESIGN.....	19
4.1	SMN	19
4.2	NUMBER OF PORTS AND SUBSYSTEMS.....	19
4.3	IFOE FAILOVER.....	19
4.4	HOST ACCESS.....	19
4.5	DEPLOYED SYSTEM CONFIGURATION	20
4.6	GTA CONTROL.....	20
4.7	GTA TEMPERATURE.....	20
4.8	MCA	20
4.9	INTERRUPTS.....	20
4.10	TELEMETRY MEMORY	20
4.11	INTERACTIONS WITH OTHER MPS	21
4.11.1	MP0 – ASP	21
4.11.2	MP1 - PM.....	21
4.11.3	MPIO.....	21
4.12	NETWORK PORT NUMBERING	21
5	FIRMWARE.....	22

MP IFOE ARCHITECTURE DRAFT 1K

5.1	BOOT, RECOVERY AND UPDATE	22
5.1.1	SYSTEM CONFIGURATION	22
5.1.1.1	MP_IFOE ALLOCATION	24
5.1.1.2	ACTIVE IFOE SUBSYSTEMS.....	24
5.1.1.3	NETWORK PORT CONFIGURATION	25
5.1.1.4	NETWORK PORT MAC ADDRESSES.....	25
5.1.1.5	IFOE SUBSYSTEM CONFIGURATION	25
5.2	SECURITY AND CONFIDENTIALITY	25
5.2.1	IFOE	26
5.2.1.1	MASTER KEY PROVISIONING	26
5.2.1.2	KEY ROLLING IN HARDWARE	26
5.2.1.3	REKEYING	26
5.2.2	CONFIDENTIAL COMPUTE	27
5.3	SYSTEMS MANAGEMENT	27
5.3.1	CONFIGURATION	27
5.3.2	TELEMETRY	27
5.3.2.1	IFOE	29
5.3.2.2	SECURITY	29
5.3.2.3	PRIORITY FLOW CONTROL	30
5.3.2.4	EXTERNAL NETWORK	30
5.3.2.5	NETWORK PORT	30
5.3.3	RAS	30
5.3.3.1	ERROR HANDLING	31
5.3.3.2	ERROR INJECTION	34
5.3.3.3	SYNC FLOOD	34
5.3.4	GPU CONNECTIVITY.....	34
5.4	IN BAND CONNECTIVITY	35
5.4.1	PRIMARY AND SECONDARY MP_IFOE.....	35
5.4.2	FLR HANDLING.....	35
5.4.3	SERVICES.....	35
5.4.3.1	DRIVER INTERFACE	35
5.4.3.2	PORT ADDRESSING	37
5.4.3.3	ENUMERATE IFOE INSTANCES.....	37
5.4.3.4	GET IFOE DETAIL.....	37
5.4.3.5	ASSIGN SECURITY KEY	37
5.4.3.6	ENABLE SECURITY KEY	37
5.4.3.7	ENUMERATE PORTS	38
5.4.3.8	GET PORT DETAIL	38
5.4.3.9	GET LINK STATISTICS	38
5.4.3.10	ETHERNET PACKET TRANSMIT AND RECEIVE.....	38
5.5	OUT OF BAND CONNECTIVITY	38

MP IFOE ARCHITECTURE DRAFT 1K

5.6	IFOE.....	38
5.6.1	IFOE	38
5.6.1.1	IFOE ISOLATION.....	39
5.6.1.2	IFOE LINK RESET	39
5.6.1.3	SDP DISCONNECT/RECONNECT.....	40
5.6.1.4	POWER SAVING.....	40
5.6.2	SECURITY	40
5.6.3	PRIORITY FLOW CONTROL.....	40
5.7	ETHERNET DRIVER	41
5.8	ETHERNET	41
5.8.1	MAC.....	41
5.8.2	PHY	41
5.9	INTERPROCESSOR COMMUNICATIONS.....	41
5.9.1	ASP	42
5.9.1.1	ASP - TDISP_TRANSITION_READY.....	43
5.9.1.2	ASP - TDISP_TRANSITION_DONE.....	43
5.9.1.3	ASP - ASK_TDISP_REPORT	43
5.9.1.4	ASP - ACCELERATOR_ID.....	43
5.9.1.5	ASP - IFOE_STATUS.....	43
5.9.1.6	ASP - GPU_ERROR	43
5.9.1.7	ASP - GET_VF_BDF	43
5.9.1.8	ASP - RAS_ERROR_CONTROL	43
5.9.1.9	ASP - RAS_ERROR_INJECT	44
5.9.1.10	ASP - RAS_GET_ERROR_MASK.....	44
5.9.1.11	ASP - RAS_ERROR_CLEAN	45
5.9.1.12	ASP - SET_DRAM_MEMORY.....	45
5.9.1.13	ASP - GET_VERSION	45
5.9.1.14	ASP - SET_MCA_TARGET.....	45
5.9.1.15	ASP - SET_ACTIVE_ACCELERATORS_MASK	46
5.9.1.16	ASP - SET_LOCAL_ACCELERATORS_MASK.....	46
5.9.1.17	ASP - GET_SRAM_MEMORY.....	46
5.9.2	PM	46
5.9.3	MPIO.....	47
5.9.4	MP_IFOE.....	47
6	FIRMWARE ARCHITECTURAL LAYERING	48
6.1	APPLICATION LAYER	48
6.2	MANAGER LAYER.....	48
6.3	SUBSYSTEM LAYER.....	49
6.4	DRIVER LAYER	49
6.5	HARDWARE ADAPTATION LAYER	50
6.6	OPERATING SYSTEM	50

7	FIRMWARE COMPONENTS	51
7.1	MANAGERS.....	51
7.1.1	NETWORK PORTS	51
7.1.2	IFOE	52
7.1.3	SECURITY	53
7.1.4	TELEMETRY.....	53
7.1.5	FAULT HANDLING (MCA).....	54
7.1.6	CONFIGURATION	55
7.1.7	HOST.....	55
7.2	SUBSYSTEMS.....	56
7.2.1	DMA.....	56
7.2.2	IFOE	57
7.2.3	NETPORT	57
7.2.4	TRACELOG	57
7.3	DRIVERS	57
7.3.1	IFOE RSMU	57
7.3.2	IFOE PMI	58
7.3.3	IFOE LINE INTERRUPTS	59
7.3.4	MMHUB.....	59
7.3.5	SYSHUB.....	59
7.3.6	ADMA	59
7.3.7	CRU/MAILBOX	60
7.3.8	SMN.....	60
7.3.9	TELEMETRY	60
7.3.10	RAS	60
7.3.11	IFOE CONTAINER	60
7.3.12	IFOE	60
7.3.13	SWITCH (EX)	61
7.3.14	SEC.....	61
7.3.15	PFC.....	61
7.3.16	MAC.....	62
7.3.17	UPI2.....	62
7.3.18	GTA.....	62
7.3.19	IEP.....	62
7.3.20	MCA.....	62
7.3.21	TRACELOG	62
7.3.22	POSTCODE	62
8	HOST AND DRIVER	63
8.1	HOST (PF).....	63
8.2	GUEST (VF).....	63
9	OPENS	64
10	APPENDIX A: INITIATOR IDS	65
11	APPENDIX B: IFOE SUB-SYSTEM INTERRUPT HIERARCHY	66
12	APPENDIX C: CONFIGURATION ITEMS	72
12.1	CONFIGURATOR.....	72
12.2	CONFIG PHASES	73

- 12.3 FUSE CONFIGURED 74
- 12.4 BIOS CONFIGURED 74
- 12.5 AI FABRIC MANAGER (AIFM) CONFIGURED 74
- 12.6 VIRTUAL POD MANAGER (VPM) CONFIGURED 77
- 12.7 FIRMWARE CONFIGURED 78
- 12.8 CONSTANT 79
- 12.9 TUNABLES 80
- 13 APPENDIX D: CREST TESTING 87
 - 13.1 WORKFLOW 87
- 14 APPENDIX E: TELEMETRY IDENTIFIERS..... 88

Table of Tables

TABLE 1: CHANGE LOG	9
TABLE 2: REFERENCES AND RELATED DOCUMENTS.....	9
TABLE 3: ACRONYMS AND ABBREVIATIONS.....	10
TABLE 4: XRMAC PORT MAPPING	16
TABLE 5: XRPFC PORT MAPPING	16
TABLE 6: XRPFC QUEUE MAPPING	16
TABLE 7: MCA ERROR BITS	31
TABLE 8: MCA_ADDR - DEVICE IDENTIFICATION.....	32
TABLE 9: IFOE MCA ERRORS	32
TABLE 10: EX MCA ERRORS	32
TABLE 11: XRSEC MCA ERRORS	33
TABLE 12: XRPFC MCA ERRORS	33
TABLE 13: XRMAC MCA ERRORS	34
TABLE 14: INTERPROCESSOR MAILBOXES	42
TABLE 15: ASP AND MP_IFOE MAILBOXES	42
TABLE 16: ASP COMMANDS TO MP_IFOE	43
TABLE 17: MP_IFOE COMMANDS TO ASP	43
TABLE 18: ACCELERATOR_ID PARAMETERS	43
TABLE 19: RAS_ERROR_CONTROL PARAMETERS.....	44
TABLE 20: RAS_ERROR_INJECT PARAMETERS.....	44
TABLE 21: RAS_GET_ERROR_MASK PARAMETERS.....	44
TABLE 22: RAS_ERROR_CLEAN PARAMETERS	45
TABLE 23: SET_DRAM_MEMORY PARAMETERS.....	45
TABLE 24: SET_DRAM_MEMORY RESPONSE.....	45
TABLE 25: SET_MCA_TARGET PARAMETERS.....	46
TABLE 26: SET_ACTIVE_ACCELERATORS_MASK PARAMETERS	46
TABLE 27: SET_LOCAL_ACCELERATORS_MASK PARAMETERS	46
TABLE 28: GET_SRAM_MEMORY RESPONSE.....	46
TABLE 29: PM AND MP_IFOE MAILBOXES.....	46
TABLE 30: PM COMMANDS TO MP_IFOE.....	47
TABLE 31: MP_IFOE COMMANDS TO PM.....	47
TABLE 32: MPIO AND MP_IFOE MAILBOXES	47
TABLE 33: MPIO COMMANDS TO MP_IFOE	47
TABLE 34: MP_IFOE COMMANDS TO MPIO	47
TABLE 35: MP_IFOE MAILBOXES	47
TABLE 36: TELEMETRY DRIVER API	60
TABLE 37: IFOE SUBSYSTEM INITIATOR VALUES	65
TABLE 38: PMI INTERRUPT PIN ALLOCATION	66
TABLE 39: CONFIGURATOR TYPES.....	72
TABLE 40: IFOE CONFIGURATION PHASES	73
TABLE 41: FUSED CONFIGURATION ITEMS.....	74
TABLE 42: BIOS CONFIGURATION ITEMS.....	74
TABLE 43: AIFM CONFIGURED ITEMS.....	77
TABLE 44: VPM CONFIGURED ITEMS.....	78
TABLE 45: FIRMWARE CONFIGURED ITEMS	79
TABLE 46: CONSTANT CONFIGURED ITEMS	80
TABLE 47: TUNABLE CONFIGURED ITEMS	86

Table of Figures

FIGURE 1: AI DATACENTRE NETWORK 12

FIGURE 2: 72 LANE NODE TOPOLOGY 13

FIGURE 3: 96 LANE NODE TOPOLOGY 13

FIGURE 4: IFOE SYSTEM 14

FIGURE 5: VIRTUALISED CONFIGURATION FLOW..... 23

FIGURE 6: BARE METAL CONFIGURATION FLOW 24

FIGURE 7: STATISTICS UPDATES 28

FIGURE 8: MCDI MESSAGE FORMAT 36

FIGURE 9: MCDI REQUEST, RESPONSE & ERROR FORMAT 37

FIGURE 10: FIRMWARE LAYERS 48

FIGURE 11: GENERAL ARRANGEMENT OF SOFTWARE COMPONENTS 51

FIGURE 12: NETWORK 52

FIGURE 13: IFOE 53

FIGURE 14: TELEMETRY 54

FIGURE 15: TELEMETRY SET ARRANGEMENT 54

FIGURE 16: ERROR MANAGEMENT 55

FIGURE 17: HOST 56

FIGURE 18: TOP LEVEL INTERRUPT HIERARCHY 66

FIGURE 19: EX INTERRUPT HIERARCHY 67

FIGURE 20: IFOE INTERRUPT HIERARCHY 67

FIGURE 21: MXH INTERRUPT HIERARCHY 68

FIGURE 22: MXRT INTERRUPT HIERARCHY 68

FIGURE 23: SD_CTRL INTERRUPT HIERARCHY 69

FIGURE 24: PFC INTERRUPT HIERARCHY 69

FIGURE 25: RSD INTERRUPT HIERARCHY 70

FIGURE 26: RSE INTERRUPT HIERARCHY 71

1.1 Change Log

Date	Owner	Version	Change details
2024-08-01	ALJ	A	First draft
2024-08-21	ALJ	B	Updates after review
2024-09-02	ALJ	C	Updates after review
2024-09-23	ALJ	D	Add architecture diagrams
2024-09-30	ALJ	E	Updates after review
2024-11-06	ALJ	F	Add details of other processors, clarify some wording
2024-11-06	ALJ	1	Clarify other processors' roles.
2024-11-09	ALJ	1A	Add details on EX, MAC and telemetry. Constraints on statistics registers.
2024-12-12	ALJ	1B	Add details on ADMA, channels
2024-12-16	ALJ	1C	Rearrange detail in sections, add TDISP commentary, correct DMA usage
2025-01-19	ALJ	1D	Add details of PMI interrupts and interrupt hierarchies
2025-01-30	ALJ	1E	Change security level, amend interrupt handling, interprocessor communication
2025-02-14	ALJ	1F	Add more telemetry detail
2025-03-04	ALJ	1G	MAC address, configuration items, notes on CReST testing
2025-04-08	ALJ	1H	Add interprocessor mailbox and command details
2025-05-15	ALJ	1I	Add GTA temperature support, comment on network port numbering, add configuration flows, suggested GPU connectivity checks.
2025-07-14	ALJ	1J	Correct change log years. Add detail on MCA handling. Update PMI->RAS interrupt handling
2025-07-28	ALJ	1K	Refined MCA handling

Table 1: Change log

1.2 References & Related documents

Document	Details
EX	EX specification
IFoE	IFoE architecture
HEADERS	(Pensando) packet headers
RSMU	Remote SMU
SECIP	SECIP3.0 MP Micro-Architecture Specification
Tracelog	AMD Distributed MPx Trace Log
XN-202465-PS	MCDI Design for MP-IFoE
XN-202493-PS	Host Networking Design for MP-IFoE
XRMAC	XRMAC specification
XRPF	XRPF specification
XRSEC	XRSEC specification

Table 2: References and related documents

1.3 Glossary

Document	Details
AICC	AMD Instinct Cluster Controller (manages Scale Out network)
AIFM	AMD Instinct Fabric Manager (manages Scale Up network)
ANLT	Auto Negotiation and Link Training
BER	Bit Error Rate
BMC	Baseboard Management Controller
CAVP	Cryptographic Algorithm Validation Program
CRC	Cyclic Redundancy Check
CRU	Configuration Register Unit

CReST	Chip Resident Self Test
CSR	Configuration Status Register
DSM	Device Security Manager
EX	Ethernet Switch
FEC	Forward Error Correction
FLR	Function Level Reset (PCIe)
IEP	Integrated Endpoint
IFoE	Infinity Fabric over Ethernet
KPX	Kompressed Packet Xbar (sits between PHY and PCS)
LLDP	Link Layer Discovery Protocol
MAC	Media Access Control
MAS	Micro Architecture Specification
MCA	Machine Check Architecture
MCDI	Management CPU Driver Interface
MDI	Media Device Interface
MID	Multimedia and IO Die (PCIe, Display and Video)
MP	Microprocessor
nBIF	New Bus Interface
nHT	Network Hyper Transport
PCP	Priority Control Point (maps to a priority level, part of VLAN tag)
PF	Physical (PCIe) Function
PMA	Physical Media Attachment
POST	Power On Self Test
RSMU	Remote System Management Unit
SDP	Scalable Data Port
SMN	System Management Network
TCB	Trusted Computing Base
TDI	TEE Device Interface
TEE	Trusted Execution Environment
TLB	Translation Lookaside Buffer
UPI2	Universal PHY Interface, version 2
VF	Virtual (PCIe) Function
VPM	Virtual Pod Manager
XGMI	External Global Memory Interconnect

Table 3: Acronyms and abbreviations

2 Introduction

This document describes the architecture of the firmware operating on the Infinity Fabric over Ethernet (IFoE) processor (MP) within the MI450 series accelerator. The MP_IFoE manages the IFoE subsystems within a MID of the MI450. The IFoE system is responsible for the scale-up network ports that use IFoE links to connect different MI450s together through Ethernet switches.

This document expects readers to be familiar with the basic design of the MI450 and its components. Brief overviews of components are given where relevant.

3 System overview

This section provides an overview of the MI450 system and the components in the IFOE system. The system architecture using MI450s is shown in Figure 1.

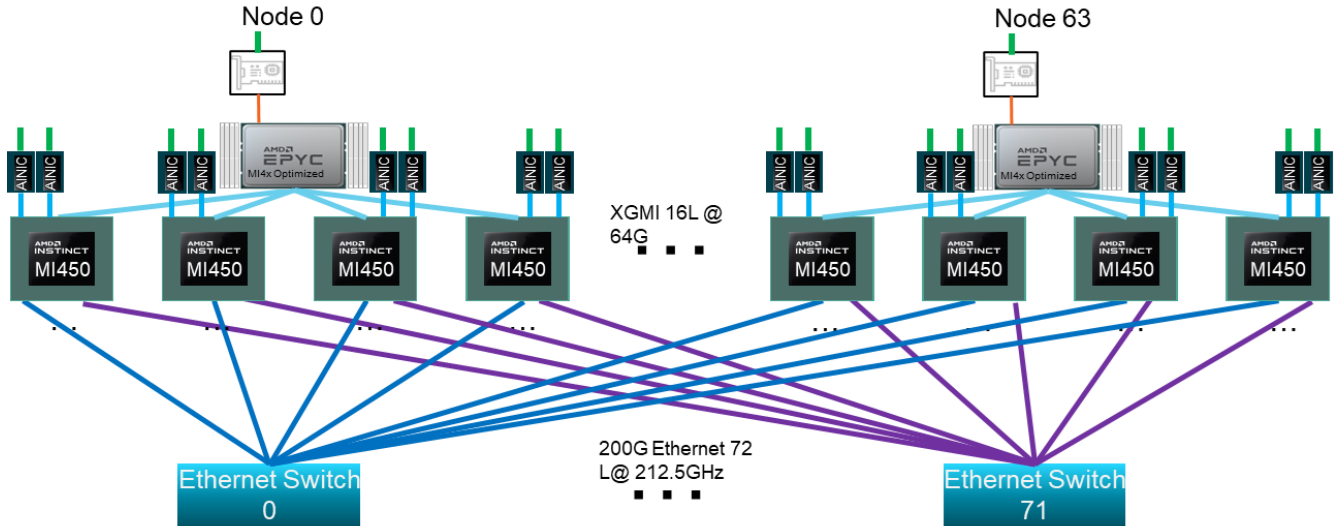


Figure 1: AI datacentre network

Earlier accelerators used XGMI to interconnect the accelerators. MI450 changes this to use IFOE which provides greater bandwidth, link failover and (optional) end-to-end encryption. The key advantage is that this is a switched network (unlike MI300 which used a fully connected mesh). This allows larger ScaleOut networks including scaleup between multiple servers (which MI300 didn't support using XGMI). The MID, of which there are two within a MI450, provides the I/O connectivity for the GPU chiplets. Each MID contains 18 IFOE subsystems with each subsystem giving either 1 x 800Gbps, 2 x 400Gbps or 4 x 200Gbps network connections.

One node within the system may be configured as shown in Figure 2: here this node¹ is configured for 72 lanes.

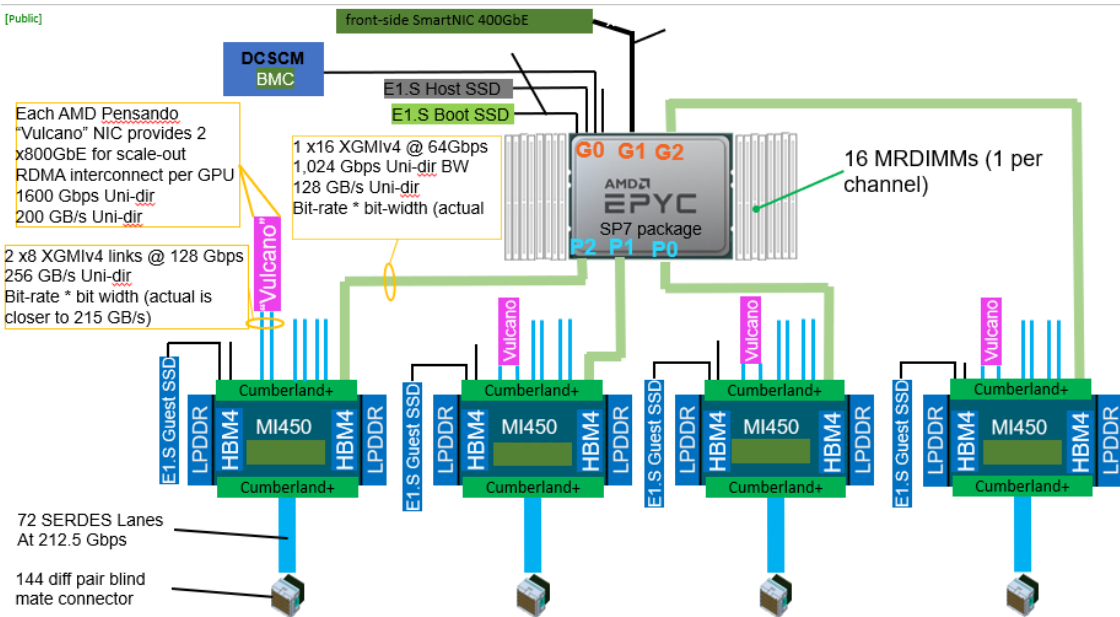


Figure 2: 72 lane node topology

An alternative node arrangement in shown in Figure 3, here this node is configured for 96 lanes (2 x 48 lanes for each MID).

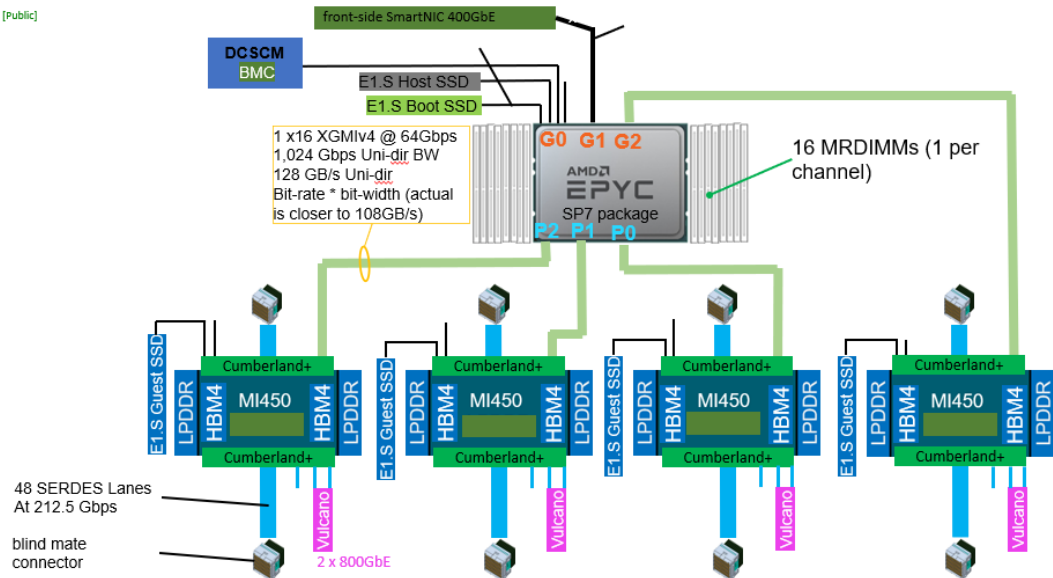


Figure 3: 96 lane node topology

¹ "Cumberland+" is the codename for the MID.

A block diagram of the IFOE system, within the MID, is shown in Figure 4. Note that there are two MIDs within a MI450 (which are connected via the SMN).

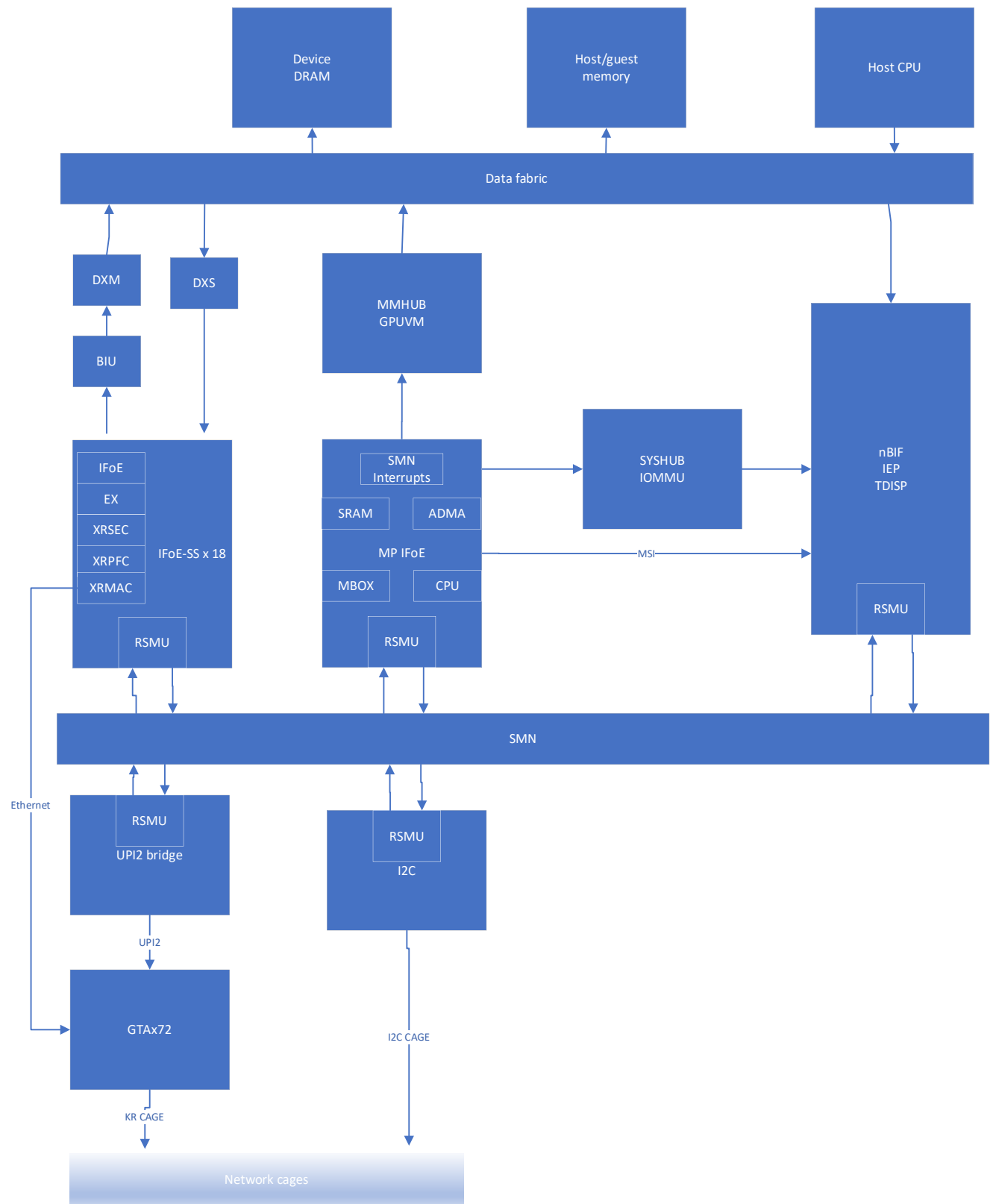


Figure 4: IFOE system

The following sub-sections describe the roles of each of the functional blocks that are managed by the MP_IFoE.

3.1 Common components

3.1.1 SMN

The System Management Network is a packet-based bus connecting components within the SoC together operating at 400MHz. The bus, since it is shared with other components, can mean that the access to other components is relatively slow. There’s an SMN latency of ~320ns to access IFOE sub-systems within the MID (worst case is ~390ns for other components): obviously additional latencies within the blocks themselves apply.

Within the MI450, each chiplet has its own SMN and these SMNs are interconnected with each other by suitable IP blocks. This allows MI450-wide communications: there is considerably extra latency in this (MID-MID access latency is O(1µs) for example).

3.1.2 RSMU

The Remote System Management Unit (RSMU) provides access control, mailbox and interrupt services. Access control is restricted to ingress to the block in which the RSMU resides. Access control is through a system of IP block IDs and unit IDs (being an instance of a specific IP block) in conjunction with addresses. The system security controller (ASP) programs the necessary rules into the RSMU when the system boots.

3.2 IFOE subsystem

The IFOE subsystem provides the network component within the IFOE system. There are eighteen IFOE subsystems with the bottom end connecting to the “Scale Up” network and the top end connecting to the data fabric. The IFOE protocol provides reliable transport of data and is similar to but not identical with the service provided by XGMI links.

While a MID may have eighteen possible IFOE subsystems, a MID may be configured to use fewer of them. This is because the underlying GTAs may be configured to provide connectivity for PCIe or XGMI links.

Note that the IFOE subsystem provides the ability to collect coherent statistics which is to say that all statistics for each block (IFOE, XRMAC and XRPFC) are captured simultaneously. This is useful when analysing the statistics as to how a system is running. Note that the XRSEC block does not support this scheme and needs to be sampled independently.

3.2.1 XRMAC

The XRMAC block provides IEEE802.1 Ethernet port functionality in the form of MAC, PCS and FEC functionality. Within the MAC there is support for the UEC’s link level retry² which provides for fast retries of errored packets to the link partner.

The port numbering depends on the network port mode as shown in Table 4. Additionally, one or more subordinate ports are assigned which define the underlying lanes and which, from the driver’s perspective, contain that lane’s FEC statistics.

Network port mode	XRMAC port(s)	Subordinate port(s)
1 x 800G	0	0: 1, 2, 3, 4, 5, 6, 7
2 x 400G	0, 4	0: 1, 2, 3 4: 5, 6, 7
4 x 200G	0, 2, 4, 6	0: 1 2: 3

² This standard is due to be ratified shortly

	4: 5
	6: 7

Table 4: XRMAC port mapping

3.2.2 XRPFC

The XRPFC block provides priority flow control for the 800Gbps link. Priority flow control allows specific traffic classes to be paused without affecting other classes, allowing management of congestion on the network. The XRPFC block contains 8 ports which are linked to the XRMAC block. The allocation of an XRPFC port to an XRMAC port depends upon the XRMAC configuration as shown in Table 5. Either 4 or 8 priority queues can be associated with each port and the statistics for those are sampled at the same time as those of the port. The expectation is that IFOE uses 4 queues (for the purposes of Request, Response, non-IFOE and Discard).

Network port mode	XRPFC port(s)
1 x 800G	0
2 x 400G	0, 4
4 x 200G	0, 2, 4, 6

Table 5: XRPFC port mapping

XRPFC port	XRPFC queue 4 queues	XRPFC queue 8 queues
0	0 ... 3	0 ... 7
1	4 ... 7	8 ... 15
X	X×4 ... X×4+3	X×8 ... X×8+7

Table 6: XRPFC queue mapping

3.2.3 XRSEC

The XRSEC block provides encryption and decryption for the Ethernet links. A variety of security protocols are supported including IPSEC, PSP (Google and ClusterSec) and IFOE. IPSEC and PSP are not used in the MI450's MID. Up to 18k IFOE streams are supported. The security keys are programmed through register interfaces by the MP_IFoE.

3.2.4 EX

The Ethernet Switch (EX) block³ provides for multiplexing/demultiplexing packets between the IFOE block and, using the internal MR-PIO block, allows the reception and transmission of management packets (that is all non-IFOE packets) intended for the MP_IFoE. Packets are transmitted and received with additional headers (global intrinsic header, P4 header and possibly an XRSEC header on receive): these are placed before the Ethernet header. The MR-PIO block knows nothing about these additional headers and provides a very simple interface to the receive and transmit FIFOs. The MR-PIO is a very simple block and requires that all packets are read completely on receive: there's no discard facility.

³ Originally the IFOE subsystem was to be deployed in Vulcano as well and there the switch allowed packets to flow between the packet processors and the network.

3.2.5 I FoE

The Infinity Fabric over Ethernet (IFoE) block provides encapsulation and decapsulation of Scalable Data Port (SDP) traffic over Ethernet. It implements the reliable, in-order and low latency delivery of traffic between GPUs using Ethernet.

3.3 PHY

The PHY block provides physical layer connectivity for the network ports and are connected through the KPX.

3.3.1 UPI2 bridge

The UPI2 bridge provides a register-based interface to the GTA PHYs for the MP_IFoE. The communication with the GTA PHYs is over links using the UPI2 protocol (UPI: Universal PHY Interface). The UPI2 protocol is an asynchronous protocol with handshaking, only one command at a time can be issued per GTA PHY. The asynchronous nature of the bus means that response times for reads and writes are variable. Not shown, there's another interface to load the PHY firmware: that loading is handled by another MP.

3.3.2 GTA

The GTA PHY provides serialization/de-serialization of network data together with the auto-negotiation and link-training through a Microblaze processor (this is where the UPI2 commands are handled). There is one GTA per PHY lane: an IFoE subsystem supports four individual lanes. The GTA are accessed through the UPI2 bridge and the speed at which it responds to requests depends on what operations it is performing. This means that register accesses can be slow.

3.3.3 I2C (not used)

Although the MID is provisioned for an I2C interface, in the system design for the MI450, the cable connection block's details are implicit in the configuration information passed to MP_IFoE during startup so there is no need to use I2C at all.

3.4 MP_IFoE

The MP_IFoE processor provides standard processor facilities as found in other processor subsystems in the MI450 MID. Be aware that access to the SMN by the CPU and ADMA are not independent and that transactions between the two will be interleaved. The CPU does have the ability to use a separate asynchronous ("non-blocking") interface to the SMN.

3.4.1 CPU

The CPU is a SiFive E6-A processor - a 32-bit RISC-V processor family device.

3.4.2 SRAM

1MB of memory for program and data.

3.4.3 MBOX

The mailboxes are provided through the Configuration Register Unit (CRU). The CRU also provides a programmable interrupt controller and timers.

3.4.4 ADMA

The ADMA component provides a DMA controller for the efficient transfer of memory between blocks. It

allows the MP to offload heavy data movement like collecting network port statistics. ADMA supports three independent queues, and ADMA breaks up transfers into bursts of 8 x 8 bytes.

3.4.5 SMN Interrupts

Interrupts⁴ from other SMN components (like IFOE, XRMAC, etc.) are raised by the component writing to the one of three mailbox queues in the MP_IFOE's SRAM: these mailboxes are separate to the mailbox registers described in 3.4.3. The queues store an AXID "address" of the interrupting component, allowing the CPU to identify the interrupting component. Each of the queue sizes is configurable.

An interrupt delivered through the power management FIFOs is identified by the AXID and an interrupt mask. The AXID includes an "initiator ID" (see 0) which, certainly for IFOE subsystems, is disjointly numbered (so the most effective lookup strategy is likely to be a binary search). The interrupt mask, for the IFOE subsystem, allows the interrupting component (IFOE, SEC, etc.) to be identified.

3.5 Host side

The host side interfaces are effectively shared between IFOE system instances, as each MI450 contains two MIDs.

3.5.1 nBIF

The Bus Interface (nBIF) block lies between the PCIe bridge and the host. This allows the MID to access host memory.

3.5.2 IEP

The integrated endpoint (IEP) provides a single Physical Function (PF) and a single Virtual Function (VF) for all the IFOE system instances in the MI450 to communicate with the host. Although there's an IEP interface in each MID, only one is active in the MI450. Each function also has an MSI interrupt that can be raised by the MP_IFOE. The MP_IFOE does not get PCIe state changes (like BME, FLR) directly from the hardware: these are proxied through other MPs in the system.

⁴ The documentation refers to these as "Power Management Interrupts"

4 Requirements arising from system design

This section briefly discusses requirements arising from the system design that need particular care in implementation and the firmware architecture. These requirements are discussed further and addressed in later sections of this document.

4.1 SMN

The SMN is, from the point of view of the CPU and ADMA, a shared resource. As noted earlier, transactions to it may be interleaved. This will affect access times to other peripherals which needs to be considered in the firmware design decisions. While there is a 16 deep FIFO associated with the SMN access, the firmware needs to avoid filling to that level.

Additionally, only part of the SMN's address space is exposed into the MP_IFoE's address space through a set of TLBs. There are a limited number of TLBs but many SMN components to access. Fortunately, each IFoE subsystem is the same size as that of a TLB: the MP_IFoE firmware allocates appropriate TLBs at start-of-day for this purpose. Other SMN accesses may require dynamic allocation although this should be avoided wherever possible.

The slowness of the SMN bus means that the CPU should avoid programmed IO for non-trivial amounts of data and, instead, rely on ADMA. This is also important because if the CPU blocks it is unable to perform any other useful work.

4.2 Number of ports and subsystems

From the preceding section, it can be seen that there are a large number of network ports and IFoE subsystems to be managed. This means that the MP_IFoE's control paths to them need to be both efficient and fair. For example, the number of statistics that can be generated from the various blocks is significant both in terms of memory required and the time required for their collection over the SMN bus. ADMA provides a convenient way of transferring data without CPU involvement.

The large number of elements to be controlled, up to 18 IFoE subsystems and 72 GTAs, combined with the SMN bus access restrictions means that polling blocks for errors, changes of state, etc. is likely to be infeasible and not provide the necessary system response times. That means wherever possible, the firmware should use interrupts to manage the peripherals.

4.3 IFoE failover

A further requirement arises from the handling of failover between network ports on an IFoE subsystem. Ultimately, this is because the host transactions are for memory transfers, and this give rise to a hard real time requirement on the MP_IFoE for port changeover. It is anticipated that a transaction timeout may be O(80ms) but the time of (failed) retransmissions must be deducted from this.

4.4 Host access

On any MI450, there's one nBIF and one IEP providing a PF (host) and a VF (guest). If two IFoE systems are active, a protocol is required to allow those instances either to share those resources directly or to allow one to become the main controller and liaise with the other. The second IFoE system can still access host memory through its nBIF but the associated IEP is disabled.

As with the SMN, host memory address space is exposed through TLBs within the MMHUB and SYSHUB, and this will require that the MP_IFoE manage those dynamically. It may be possible for them to be set up statically as there are a limited number of targets unlike the SMN.

4.5 Deployed system configuration

The system configuration determines the role of specific elements in the MID: the role may be IFoE, PCIe or XGMI. That configuration needs to be known by the MP_IFoE, in conjunction with the cable connection module, for proper operation of the system. The IFoE port numbering needs to be consistent throughout the system to avoid misconnections and best performance from the available Ethernet links. This configuration information needs to be passed to the MP_IFoE at boot time.

4.6 GTA control

The IFoE's GTAs are controlled through UPI2, which is an asynchronous register interface. This complicates the firmware required to support the GTAs because of the number of GTAs in the system. The simplest solution is to simply wait, but this would result in very uneven service times and the likelihood of unreliable system operation. Another simple solution might require a thread per GTA but there's insufficient system memory to support that. This means that a more complicated solution is needed to handle the asynchronous nature of the interface involving interrupts, state machines and the caching of register values.

4.7 GTA temperature

The GTA requires regular updates of the local temperature (where local is the nearest sensor to the GTA in the chip). This is so that the GTA firmware can correct coefficients for thermal drift. The temperatures need to be collected from the MPIO (or PM) processors and programmed into the PHY.

TODO: At the time of writing the update period is not known

4.8 MCA

Normally each IP block provides MCA capabilities through a standardised MCA block. However, the IFoE subsystems don't contain such a block although they do provide MCA like functionality through their register interfaces. There is a MCA block, present within the SRAM of the MP_IFoE tile, providing handling for hardware errors within the tile. There are 38 hard errors and the design of the MCA IP means that nominally there are 25 available for firmware use. However, MP_IFoE is configured for a total of 50 errors, so support for 12 soft errors is possible: the number of ("MCA") errors from the IFoE subsystems is much larger than that. Additionally, when reporting errors an indication of the subsystem's identity needs to be reported – this can be the initiator ID of the subsystem. That and the actual error source can be presented as part of the MCA_ADDR register (which is 64 bits) – none of the IFoE elements have such large addresses. This allows the origin of faults to be reported precisely as required for RAS reporting.

4.9 Interrupts

The processor is a Risc-V E6.A processor which has a programmable interrupt controller (PIC), a platform interrupt controller (PLIC) and a core local interruptor (CLINT)⁵. From a processor perspective these appear as hierarchical interrupts from the root interrupt controller within the processor. This necessitates the use of Zephyr's multi-level interrupt support but, significantly, takes all of the available interrupt levels. Hence handling of IFoE subsystem derived interrupts requires a separate framework because there's a reasonably deep hierarchy associated with that.

A further issue with interrupts is that the top-level ("group") interrupts from an IFoE subsystem cannot be disabled and are enabled by default.

4.10 Telemetry memory

The number of IFoE subsystems and the volume of statistics that can be produced by them means that the

⁵ In theory, the PIC and PLIC could be combined into one driver with some further development work to free up one level of interrupt hierarchy.

basic SRAM of MP_IFoE is insufficient to hold them. For this reason, an area of GPU memory is required to hold those statistics. The region needs to be sufficient to hold the information produced by both MP_IFoEs in an expanded MI450 configuration. The memory should be 1MB in size, which is divided equally between the MP_IFoEs and is referred to as GPU_telemetry. The telemetry memory becomes unavailable when there is a serious error affecting the data fabric or system (“sync flood”).

4.11 Interactions with other MPs

The MP_IFoE does not exist alone within the MID and interacts with other processors in it. This section discusses those other processors with which the MP_IFoE interacts.

4.11.1 MPO – ASP

The ASP is the security processor for the system. Communication flows between ASP and MP_IFoE are required to support the system’s Confidential Computing requirements (for instance error in a TDI). Configuration in the form of the system’s topology table needs to be passed to the MP_IFoE so that the IFOE instances can be configured correctly. The MP_IFoE needs to track changes in the TDI’s state as well as report attempts to change the configuration when it is locked so that the DSM can correctly transition the device to the ERROR state. The MP_IFoE doesn’t participate in the actual TDISP protocol, which is handled entirely by the ASP in its role as DSM.

Software protocols are required between the two processors.

4.11.2 MP1 - PM

MP1 handles PCIe events generated by the IEP/nBIF. This includes, but is not limited to, FLRs. Some PCIe events affect the operation of the IFOE system (for example FLR clears keys within the IFOE sub-systems).

Software protocols are required between the two processors.

4.11.3 MPIO

MPIO configures the KPX and PHYs at start of day using the system’s topology table. The configuration selects whether the IFOE subsystem is enabled or the PHYs are configured for XGMI use. For the PHYs this involves loading the relevant firmware image and bringing the PHY to an initial idle state. There’s no requirement for MPIO and MP_IFoE communications relating to this at present but the MP_IFoE can only address the IFOE subsystems when the MPIO has completed its configuration: MPIO indicates success or failure of that to the ASP.

MPIO⁶ handles out-of-band telemetry and feeds such data back to the BMC for system management purposes. The MP_IFoE generates a substantial amount of telemetry data and the efficient transfer of such data is required. The MP_IFoE gives a reduced set of telemetry data to the MPIO and there is always available irrespective of whether the MP_IFoE can access the telemetry data stored in the “telemetry memory” or not.

Software protocols are required between the two processors.

4.12 Network port numbering

The variety of configuration options for an IFOE system means that it is important to have a uniform scheme for describing network ports. Since the configuration is fixed through the BIOS configuration, the simplest scheme is to have N network ports where N is the number of configured IFOE subsystems (across both the primary and secondary MIDs) multiplied by the number of active network ports on each subsystem (so 1, 2 or 4). Mapping functions are necessary to translate to a specific IFOE subsystem.

⁶ MP1 can also handle telemetry but over I2C which is too slow for the volume of telemetry data generated

5 Firmware

5.1 Boot, recovery and update.

The system MP (ASP) manages the verification and loading of any firmware images required for the MP_IFoE. There is no expectation that the MP_IFoE is involved in the update of any firmware images other than as the user of the updated image. Furthermore, there is **no** expectation of “impactless” updates: all updates are impactful. This is a practical decision based on the hardware design, the amount of memory available and the information that would have to be saved to achieve impactless update.

The boot time constraints have yet to be determined but the time to full operation is necessarily constrained by the time to establish any network links as well as configuration of the IFOE parameters in the IFOE subsystems.

When the MP_IFoE boots and has initialised all its drivers, subsystems and managers, it indicates this to other MPs through the MPIFOE_FIRMWARE_FLAGS register. POST codes are presented through a standardised interface for diagnostic purposes.

5.1.1 System configuration

The MID’s architecture is very flexible but the running configuration is set when the system boots: there’s no expectation of changing the configuration while the system is running. There’s a need for the MP_IFoE to report back to the system controller that it is ready for operation based on that configuration (or that the configuration is incompatible).

The general flow of configuration is shown in Figure 5 and Figure 6, also see Appendix C: Configuration items.

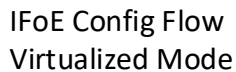


Figure 5: Virtualised configuration flow

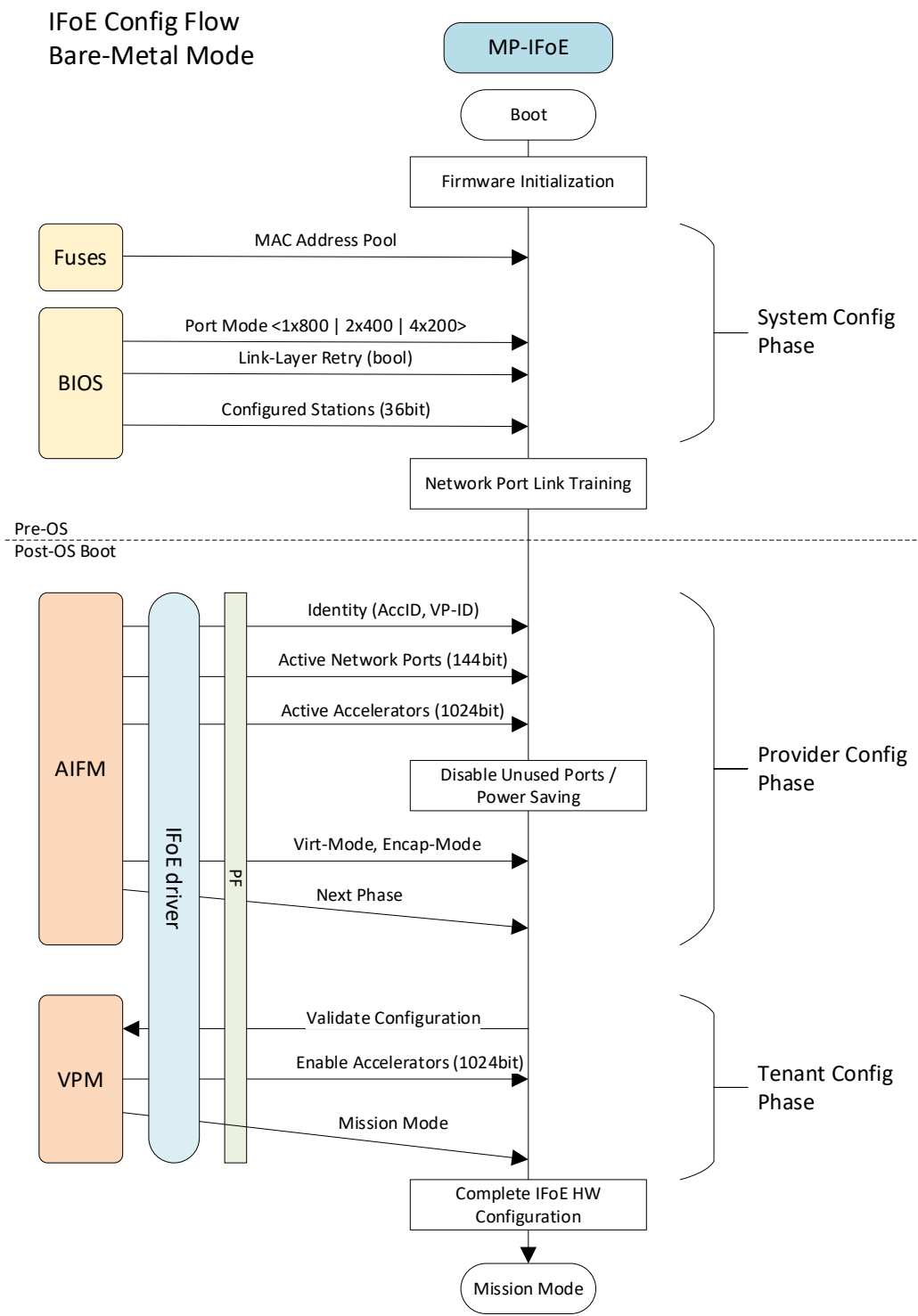


Figure 6: Bare metal configuration flow

5.1.1.1 MP_IFoE allocation

When two MIDs are present in the system, one MP_IFoE needs to be designated as the primary MP_IFoE (through which host communications flow initially) and the other designated as the secondary MP_IFoE. The detection of an MP_IFoE’s role is determined by reading the SMUIO_MCM_CONFIG register at start of day.

5.1.1.2 Active IFoE subsystems

The principal configuration detail that the MP_IFoE requires is the number of IFoE subsystems available to

it. The configuration will depend upon the system configuration. The number of IFOE subsystems selected as active needs to be consistent with the information read from the cable connection block⁷ – if there's no cable connection for a given IFOE subsystem then that subsystem will be marked as inoperable. Note that even if an IFOE subsystem is not configured in the system configuration, it is still present – but has no communication capabilities (the top and bottom links are disconnected).

5.1.1.3 Network port configuration

Once the basic IFOE configuration has been established, the configuration of the ports within an IFOE subsystem needs to be established because the network port manager needs to know whether the ports are to be configured as 1x800G, 2x400G or 4x200G. This configuration is also important for how the Ethernet links are connected to the connecting Ethernet switches.

5.1.1.4 Network port MAC addresses

Part of the configuration detail required for the network ports is that of the MAC addresses assigned to the ports. This detail is necessary not only for basic connectivity but also for IFOE operation: in some configurations the IP addresses are required too.

The addresses can be divided into two use cases: those required for IFOE operation and those required for out-of-band management.

A straightforward way of generating all the required IFOE MAC addresses is to use the GPU's identifier in conjunction with the IFOE subsystem initiator, the MID number and the network port number. These would be combined in a local MAC address. Such a scheme simplifies the configuration of the system and avoids one GPU having to obtain all the other GPU's MAC addresses.

The out-of-band management MAC addresses are programmed at manufacture and can be read from the system fuses.

5.1.1.5 IFOE subsystem configuration⁸

Further IFOE configuration information consists of the packet format of the IFOE packets on the wire. Again, this needs to be configured uniformly across the system.

Further configuration is required with relation to the identity of the GPU and the connected GPUs.

5.2 Security and confidentiality

TODO (Security still being defined)

The MP_IFOE is within the secured MI450 system and additionally within the trusted computing base (TCB). While the MP_IFOE does not participate in any security management directly, it can provide information back into the security framework, which might include "measurements" in support of the security requirements⁹. That said, the MP_IFOE does have responsibility for programming the keys used in IFOE operations as detailed in the next section.

The separation of host and guest is required by the security policy in support of confidential computing. Although only limited amounts of host memory are exposed to MP_IFOE for host and guest roles (packet buffers, MCDI buffers and telemetry buffers), care should be taken in designing firmware that any internal "shared" memory is scrubbed (zeroised) and that firmware design follows best security practices.

⁷ Unless the configuration includes detail on the connection block.

⁸ May be handled by AIFM dynamically.

⁹ As yet, the system security requirements are undefined.

5.2.1 IFOE

Some care is required within the system for the handling of keys that are to be applied to the IFOE's streams. The mechanism for transferring these keys is likely to be through the AI Fabric Manager (AIFM). IFOE supports optional encryption of network traffic, providing strong confidentiality and integrity guarantees. Encryption is end-to-end, and therefore the interconnecting switches are not part of the Trust Computing Base (TCB) when encryption is enabled.

IFOE supports many transport streams divided into two groups, one group to carry host transactions and another for guest transactions. Each group has a pair¹⁰ of master keys: this ensures separation between hosts and guests. It also allows tenants to manage their own keys. Individual streams within a group are encrypted/decrypted with a key derived from the appropriate master key, using other stream specific information.

Host and guest keys are programmed, using the AIFM, through the appropriate driver: PF for host and VF for guest. This separation supports the security policy.

Basic assurance of an IFOE's XRSEC cryptographic functionality is provided by NIST's Cryptographic Algorithm Validation Program (CAVP) support within the hardware. This gives assurances about the correct operation of the security hardware. CAVP tests are run at start of day on each XRSEC to ensure that the block is operating correctly: the XRSEC is unusable before the CAVP tests have run successfully¹¹.

5.2.1.1 Master Key Provisioning

Before secure IFOE transactions can take place, the master encryption keys need to be distributed to all the IFOE subsystems within the virtual POD. At the start of the day, there are no keys in operation, so unlike rekeying, there's no need to disable old keys; merely enable the keys. Each GPU contains several IFOE subsystems, and there is a distinct master key for each of those subsystems. Those keys need to be programmed into the appropriate subsystem of each GPU (master-N goes to IFOE-N). The provisioning flow for a given IFOE subsystem is as follows:

- The new key is passed to all GPUs in a virtual POD
- The GPUs program the new key into the IFOE subsystem, enabling the key for receive and transmit

The AIFM can manage this flow, ensuring that all entities are updated before declaring the system ready for operation.

5.2.1.2 Key Rolling in Hardware

Within a group, key rolling is implemented to prevent side-channel attacks and limit damage by broken keys in a group (host or guest). The encryption scheme uses encryption keys calculated using a key derivation function with input data being taken from the (IFOE) packet header. This avoids issues with side-channel attacks like differential power analysis (DPA) if a single encryption key were used as it minimizes the power traces generated by the shared key and makes it harder for any attacker to break the cipher key for an IFOE group.

5.2.1.3 Rekeying

Each IFOE subsystem allows two master keys per group. Periodically, for best security practices, the key in use needs to be replaced. An IFOE subsystem allows two active decryption keys and one active encryption key. Clearly, rekeying requires high-level coordination between the GPUs. The basic rekeying flow is as follows:

- The new key is passed to all GPUs in a virtual POD

¹⁰ Two master keys are needed to support online re-keying.

¹¹ Standard security practice. Execution time as yet unknown but expected to be short.

- The GPUs program the new key into the IFoE subsystem, enabling that key for reception
- The GPUs are told to change keys
- The GPUs enable the new key for transmission in the IFoE subsystem
- The GPUs are told to retire the old key, which disables the original receive key in the IFoE subsystem

During the transition window, the receiver can accept both the old and the new keys. This flow can be managed by the AIFM and is, as can be seen, similar to but not identical with the initial master key provisioning flow.

5.2.2 Confidential compute

The IFoE system is part of the Confidential Compute solution for the MI450. When using the VF, it operates as a TDI and changes that affect the security of that TDI need to be notified to the DSM (the ASP). Similarly, the DSM needs to respond to changes in the GPU's TDI (since the two TDIs are effectively linked).

5.3 Systems management

Systems management provide an important part of the Reliability, Availability and Serviceability (RAS) for the MI450 system, allowing effective management and control of devices. RAS is concerned with on chip errors whereas as telemetry is concerned with information on the data flows within and without the system.

5.3.1 Configuration

IFoE configuration information consists of the packet format of the IFoE packets on the wire. Again, this needs to be configured uniformly across the system.

Further configuration is required with relation to the identity of the GPU and the connected GPUs.

The expectation is that the AIFM handles the configuration through the host (PF & VF) interface.

5.3.2 Telemetry

The MP_IFoE provides telemetry for each IFoE subsystem. Most hardware statistics are accumulated periodically in RAM-based 64-bit counters. The individual hardware blocks maintain counters at a lower resolution and, given the likely traffic flows, it is important that they do not overflow in normal operational timeframes. The amount of telemetry data is significant because of the number of components and therefore the accumulated statistics are held in memory external to the MP_IFoE since that doesn't have sufficient space in its own memory. Updating the statistics involves:

- Read (and clear) statistics in hardware block
- Read stored statistics from external memory into "cached" statistics
- Update "cached" statistics
- Write cached statistics back to external memory

The general scheme is shown in Figure 7.

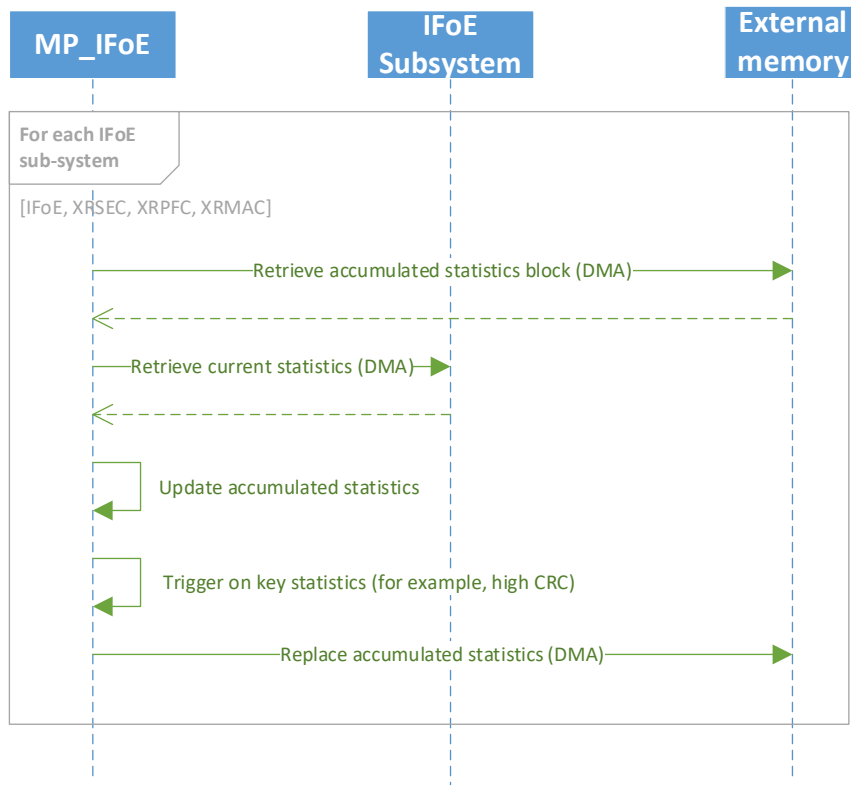


Figure 7: Statistics updates

Storing the statistics in external memory provides the possibility, given suitable security permissions, of the system BMC obtaining statistics out-of-band directly. Clearly, the host may also require in-band knowledge of these statistics which may be best accommodated by duplicating some of the statistics.

One limitation of holding statistics in external memory is that if the data fabric enters the “sync flood” (fatal error) condition, that memory becomes unavailable. This requires that some key statistics are held locally for out-of-band transfers in such circumstances (so that diagnostic information can be retrieved).

The telemetry information captured needs to be presented to different clients in the system:

- Host (via PF)
- Guest (via VF)
- BMC (via MPIO)

The volume of statistics generated by the IFOE subsystems means that it is infeasible for each client to have its own set of statistics – that would require much more manipulation of the figures by the MP_IFoE. Instead, a client selects which blocks of statistics they wish to observe. Not all clients are permitted to access all blocks so, for instance, the guest would not get network related statistics. The blocks available are:

- IFOE
- External (EX)
- Security (XRSEC)
- Priority Flow Control (XRPFC)
- Network port (XRMAC)
- Derived (for example link statistics)

Each block is uniquely numbered as are the individual statistics within each block (see Appendix E:

Telemetry identifiers). The numbering is immutable and readable by the client – so that they can determine what values are being returned. This provides a future-proof, expandable system for reporting the statistics. Clients cannot choose which statistics they get from a block (it is all or nothing): this avoids unnecessary work on the part of the MP_IFoE and reduces pressure on transfers across the data fabric. Given that there are multiple clients, the ability to reset statistics is not permitted (or at the very least is restricted to specific clients). Some blocks of statistics may be restricted to maintain the system's Confidential Compute requirements: this is because such information could leak detail about a guest's use of the GPU.

Statistics are captured synchronously for an IFOE subsystem – a common (hardware) clock drives the capture of the statistics from most components within a subsystem. This permits a coherent set of statistics to be captured. However, it is important to note that the statistics registers are not contiguous within a block's CSR address space and scattered within it as multiple small blocks (both IFOE and XRPFC are cases in point). This means that the DMA configuration is more complicated than it might first appear – it isn't possible to read "over" gaps in the CSR addresses because that provokes hardware errors.

5.3.2.1 IFOE

The IFOE statistics include detail of what packets are being processed and how.

- Packets opened
- Packets closed
- Evictions due to MTU
- Evictions due to force
- Evictions due to timer expiry
- Evictions without forced messages delayed by chain
- Evictions with forced messages delayed by chain
- Evictions that reach MTU threshold due to the scheduler acknowledgement
- Evictions with forced messages delayed due to the scheduler acknowledgement
- SDP messages
- SDP messages with force
- SDP messages with chain
- SDP message words
- SDP message length count (8B granularity)

These provide a view of how well the IFOE messages are packing SDP transactions.

5.3.2.2 Security

Directed security statistics are available for a limited number of security associations (SAs): those require specific configuration. The most likely use is checking the behaviour of a particular stream. General statistics presented include:

- Payload bytes for IFOE packets encrypted
- IFOE packets encrypted without an EOP error
- IFOE packets encrypted with an EOP error
- Payload bytes for IFOE packets decrypted
- IFOE packets decrypted successfully
- IFOE packets decrypted unsuccessfully (failed validation)
- IFOE packets which are processed and include framing error or EOP error

5.3.2.3 *Priority flow control*

The priority flow control statistics include:

- Per queue transmit counts (bytes, packets)
- Per queue receive counts (bytes, packets)
- Per queue TX periods
- Per queue RX periods
- Per priority PFC transmitted counts
- Per priority PFC received counts

5.3.2.4 *External network*

These statistics include local (non-IFoE) packet statistics:

- Unicast packets received
- Multicast packets received (e.g. LLDP)
- Packets dropped
- Packets transmitted

5.3.2.5 *Network port*

Each network port captures telemetry data describing Layer 2 (L2) MAC and FEC statistics as well as link state and stability.

The L2 statistics include:

- Receive packet histogram
- Transmit packet histogram
- Packet errors
- Byte counts
- Unicast and multicast packet counts
- FEC corrected/uncorrected counts

These provide a standard set of Ethernet statistics. Additional L2 statistics are provided for link level retries which include:

- ACKs, NAKs,
- Flush counts
- Discarded counts

These provide a good indication (together with the FEC statistics) of the local link conditions. Additional telemetry describes the link state, number of link transitions, time up, etc. The bit error rate (BER) for each lane of a network port is calculated¹² from the available FEC statistics over the last sampling interval. The BER is an important statistic in measuring the quality of the link and is supplemented by the number CRC errors occurring.

5.3.3 *RAS*

RAS protects data integrity and attempts to ensure system availability for extended periods of operation. Typically, the errors relate to single (correctable) and double (uncorrectable) ECC errors occurring within blocks but may include parity errors on busses, unexpected overflows, and so on.

¹² Note that the BER calculation breaks down when there are large numbers of uncorrected errors since they are “uncountable”. RS-FEC as deployed in 200G/400G/800G Ethernet can only correct 15 errors in a block: the number of uncorrected errors is something more 15.

5.3.3.1 Error handling

Errors within the MI450 are handled through AMD’s Machine Check Architecture (MCA). MCA provides a uniform framework for the reporting of errors with different severities allowing different actions on receipt of an error. The IFoE subsystem IP blocks don’t contain an MCA IP block (or blocks), so errors are passed up to the system through the MP_IFoE’s MCA block. Note that this requires some autonomous actions by the relevant IP blocks in terms of error containment since performance is necessarily constrained by MP_IFoE service-time latency.

MCA errors are “CPU centric” and relate to errors occurring on chip (so within the IFoE system). External errors like Ethernet CRCs or link drops are handled through in-band (host) mechanisms. The MP_IFoE’s handling of MCA errors requires proper priority categorisation to one of the four:

- Fatal. An error that cannot be corrected by hardware.
- Deferred. An error that cannot be corrected by hardware, but which do not cause an immediate interruption in operation.
- Corrected. An error that has been corrected by hardware, causing no loss of data or corruption to it.
- Transparent. An error that has been corrected by hardware and the hardware is able to correct a second error in the same component. (Essentially the same as corrected.)

The error information associated with each reported error also must include appropriate information to localise the error within IFoE system. This requires a suitable soft MCA hierarchy being developed allowing identification of IFoE subsystem, component within that subsystem and detail of the error within that component.

The MCA hardware supports two targets for error signalling: in-band (x86 host) and out-of-band (BMC). There are two sets of registers to support this (out-of-band registers are indicated by `MCA_PSP_` in the register name). When handling MCA errors, the MP_IFoE needs to process any error for both error targets: depending on what the system is doing an error may end up in just one of the targets. The following discussion only refers to the in-band register names.

MCA has a status register of 63 bits width that indicates which component is in error. Conventionally software errors are numbered downwards from bit 62. As noted in section 4.8, there are more errors from the IFoE subsystem than can be accommodated in the spare bits of the MCA register. Furthermore, the MP_IFoE is only provisioned for 50 error bits in the MI450 design. Therefore, the MP_IFoE uses the bit number in the `MCA_CTL` register to indicate the component within the system as shown in Table 7. On receiving an error the MP_IFoE sets the *ErrorCodeExt* field in the `MCA_STATUS` register.

Component	Bit
IFoE	49
EX	48
XRSEC	47
XRPFC	46
XRMAC	45

Table 7: MCA error bits

The `MCA_ADDR` register is 64 bits wide and the upper 32 bits are used to identify the detail of the element as shown in Table 8.

Component	Bit	Width	Comment
MID	48	2	0 – primary 1 – secondary

IFoE subsystem index	40	8	Index of IFOE subsystem (0 ... 17)
Error identifier	32	8	Per block error ID describing actual fault

Table 8: MCA_ADDR - device identification

The MP_IFoE follows the rules required for MCA operation in terms of maintaining error counts and reporting the oldest highest priority error. There are known race conditions associated with the MCA which need to be considered in the design which may be exacerbated with a firmware implementation.

The following tables give the “MCA” reporting elements in each block¹³. ECC errors are reported as single error corrected, double error detected (SECDED). In the tables, ErrId corresponds with the Error Identifier shown in Table 8.

Element	ErrId	Inject	Notes
ECC			
SDP RX UNPACK	0	Y	
RX DECAP	1	Y	
TX ENCAP	2	Y	
TX SCHED	3	Y	
TX RENG	4	Y	
TX WENG	5	Y	
TX BUFFER	6	Y	
SDP TX PACK	7	Y	
Fatal			
RX UNPACK parity	8	Y	
RX ACK overflow	9	Y	
TX ACK overflow	10	Y	
CON credit error	11	Y	
NET credit error	12	Y	
PORT MAP RES overflow	13	Y	
PORT MAP REQ overflow	14	Y	
TX PACK IF error	15	Y	
TX PACK IF parity	16	Y	
TX PACK IF switch	17	Y	
TX PACK IF data	18	Y	

Table 9: IFoE MCA errors

Element	ErrId	Inject	Notes
ECC			
Packet FIFO	0	N	
Fatal			
Packet control FIFO parity	1	N	

Table 10: EX MCA errors

Element	ErrId	Inject	Notes
---------	-------	--------	-------

¹³ To be completed based upon final register assignments

ECC		
TX SAD Key	0	Y
TX SAD_L Key	1	Y
TX SAD CFG Key	2	Y
TX SAD CTX Key	3	Y
TX IFoE CTX Key	4	Y
TX SB Key	5	Y
TX AL Key	6	Y
RX SAD Key	7	Y
RX SAD_L Key	8	Y
RX SAD CFG Key	9	Y
RX SAD CTX Key	10	Y
RX IFoE CTX Key	11	Y
RX SB Key	12	Y
RX SPI Key	13	Y
RX AL Key	14	Y
RX SNF Key	15	Y

Table 11: XRSEC MCA errors

Element	ErrId	Inject	Notes
ECC			
RX EOP RAM	0	Y	
RX page number RAM	1	Y	
RX data RAM0	2	Y	
RX data RAM1	3	Y	
RX data RAM2	4	Y	
RX data RAM3	5	Y	
RX data RAM4	6	Y	
RX data RAM5	7	Y	
RX data RAM6	8	Y	
RX data RAM7	9	Y	
TX SOP RAM	10	Y	
TX EOP RAM	11	Y	
TX data RAM0	12	Y	
TX data RAM1	13	Y	
TX data RAM2	14	Y	
TX data RAM3	15	Y	
TX data RAM4	16	Y	
TX data RAM5	17	Y	
TX data RAM6	18	Y	
TX data RAM7	19	Y	
Fatal			
TX overflow	20	N	
TX credit error	21	N	

Table 12: XRPFC MCA errors

Note that the XRPFC coalesces ECC errors into a single bit for each direction (TX and RX). However, it is possible to generate errors in individual blocks which is why the RAMs are identified separately. Such coalesced ECC errors are reported as EOP RAM errors for the relevant direction.

Element	ErrId	Inject	Notes
ECC			
FEC buffer0	0	Y	
FEC buffer1	1	Y	
FEC buffer2	2	Y	
FEC buffer3	3	Y	
Replay buffer	4	Y	
Replay buffer pointer (even)	5	Y	
Replay buffer pointer (odd)	6	Y	

Table 13: XRMAC MCA errors

5.3.3.2 Error injection

In order to demonstrate correct error reporting, the hardware blocks have the ability to inject errors into different areas within the IFOE subsystem. This requires an out-of-band mechanism for command and control. The component to be injected uses the same scheme for addressing as the error reporting. Note that most error injection requires that data are flowing through the block (for example, memory is being read).

5.3.3.3 Sync flood

When the system detects a critical failure, this can result in a “sync flood” which effectively shuts down the data fabric. That prevents any access to the host or device memory. This has consequences because telemetry statistics are held in external (to the MP_IFOE) memory with the effect being noticed as an error in an ADMA transfer (the CPU won’t detect it itself).

5.3.4 GPU connectivity

There is a need to maintain a view of the connectivity between GPUs within a POD. This allows the AIFM to get a quick, global, view of the system. Since the expectation is that every GPU will be accessible over every IFOE subsystem, this implies that MP_IFOE needs to check that in some way. Given the limitations of the EX block (3.2.4), a mechanism is required that does not overload the MP_IFOE on either transmit or receive of Ethernet packets.

Once the system has been suitably configured and is in the TENANT_CONFIG or MISSION_MODE states (see 12.2), the connectivity starts.

The suggested mechanism is that the MP_IFOE send a notification message to other GPU as follows:

- For each active IFOE subsystem
 - For each active GPU/accelerator that isn’t the local one
 - Send notification message containing the local GPU’s view of the target GPU’s connectivity.
 - (Pause)

This ensures that there should be fewer bursts of notifications to a given CPU. This doesn’t guarantee that such bursts aren’t possible however so there are pathological cases in which bursts can arrive. The buffering within the MRPIO block should accommodate these bursts provided that they are infrequent.

Receipt of a valid notification updates a current map of active accelerators on a given IFOE subsystem. A valid notification is one that contains the correct accelerator ID and VPOD ID. The current map updates the last map when the end of the sweep is achieved at which point the current map is zeroed. The AIFM can retrieve the connectivity map which is the logical OR of the current and last map (so the connectivity map covers up to two sweep intervals).

TODO: Optimisations are possible using knowledge of the other systems connectivity.

5.4 In band connectivity

In band connectivity is provided through the integrated endpoint present in the nBIF, which gives both host (PF) and guest (VF) connectivity. This also uses the SYSHUB to access memory on the host.

5.4.1 Primary and secondary MP_IFoE

A complication in the system design is that there is just one IEP for the system: that means that both MP_IFoEs must share that one IEP. To avoid difficulties in the direct sharing of the IEP, one MP_IFoE is designated as the primary instance and the other as the secondary instance.

The primary MP_IFoE handles all direct IEP communications and the generation of any required (MSI) interrupts. The primary MP_IFoE interprets the request to determine the destination for the request. For any secondary directed request, the primary instance forwards the received command doorbell to the secondary instance who, in turn, replies with the status. Since command doorbells describe the address of the command message in memory, this minimises the data transferred between the primary and secondary instances. Some mailboxes may map to descriptor rings (for receiving and sending Ethernet packets) rather than commands.

5.4.2 FLR handling

The IEP provides PF and VF functionality. As with all PCIe devices, these may receive function level resets (FLRs): the scope of an FLR depends on whether the PF or the VF is affected.

For a PF (host) the following actions are required:

- Stop host and guest IFoE traffic
- Clear any host and guest cryptographic keys

PF FLR affects the guest because it (effectively) contains the VF.

For a VF (guest) the following actions are required:

- Stop guest IFoE traffic
- Clear guest cryptographic keys

5.4.3 Services

In-band connectivity provides a driver interface for the following services:

- System configuration: number of network ports
- System configuration: port detail
- Assignment of security keys (where security is enabled)
- Reading of MAC address assignments
- Network port statistics
- Ethernet packet transmit and receive

5.4.3.1 Driver interface

The driver interface provided by the MP_IFoE subsystem uses MCDI (Management CPU Driver Interface), which is a synchronous request-response control interface that allows clients to send requests to MP_IFoE firmware and receive responses. MCDI can also generate asynchronous events – to indicate changes in link

state for example. MCDI has been extensively used for managing SmartNICs and network cards. MCDI is carried over a variety of transports including PFs and VFs with appropriate security separation.

MCDI defines a simple message format that has a fixed-format header followed by a request/response-specific variable length payload.

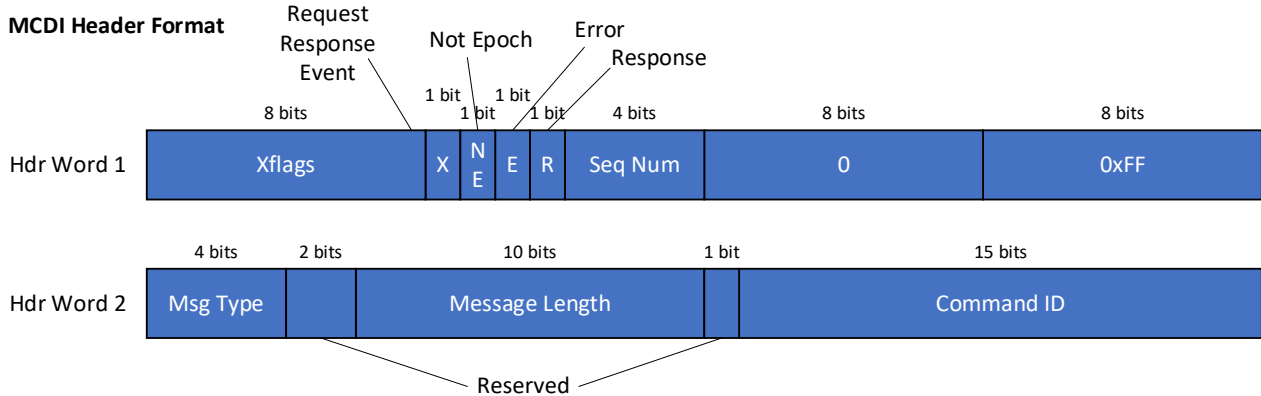


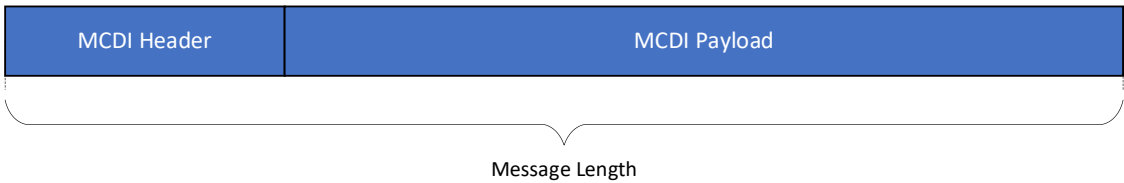
Figure 8: MCDI Message Format

Each MCDI message format consists of an MCDI Header followed by a payload. The MCDI Header is 64bits and has a fixed format with the following fields:

- **Command ID.** 15bit value identifying the MCDI command
- **Message Length.** 10bit value indicating the size in bytes of the MCDI message including the header
- **Message Type.** 4bit field used to separate different categories of MCDI commands- this is used for sets of commands that are independent from each other e.g. MCDI commands used to manage the CDX Bus are a different type from general MCDI. For all main category MCDI operations, Message Type is 0.
- **Response.** If this bit is clear the message is an MCDI Request; if set the message is an MCDI Response
- **Error.** If set, this bit indicates that an MCDI Response is an error response. For error responses, there is a fixed format
- **Not Epoch.** This field is only used for MCDI over TLP (PCIe) interface. Every client should clear this bit for first MCDI request following an NMC reboot (an MCDI Reboot event is sent to all clients when the NMC reboots) and set it thereafter. If a client does not clear the bit for the first MCDI (e.g. in the case where the client is unaware that the NMC has restarted) the MCDI request will fail with a reboot notification. Note that this field had to be introduced in a backward compatible way in an earlier generation of product which explains the inverted semantics.
- **Xflags Bit0.** Bit 0 of the Xflags field is used to request an MCDI Event to notify the client that an MCDI Request has been processed and the Response sent back to the client.
- **Sequence Number:** This can be used by MCDI clients to check that messages have not been lost. The NMC will copy the sequence number from the Request into the Response for all MCDI operations. Clients increment the sequence number for each MCDI Request and check that the response has the same sequence number.

MCDI Request and Response messages follow the same format: an MCDI header followed by a variable-length payload. The exception is errored responses. These have a specific format of an MCDI header followed by an error code followed by an optional additional info word.

MCDI Message Request & Response



MCDI Message Error Response

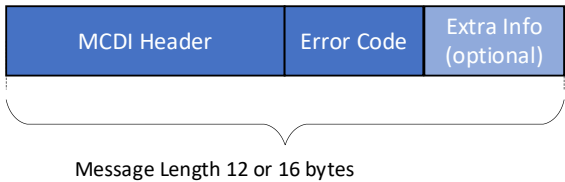


Figure 9: MCDI Request, Response & Error Format

5.4.3.2 Port addressing

The driver interface provides a very large number of network ports (potentially up to 144). These are represented as an opaque handle `PORT_HANDLE` to the driver. Specific MCDI commands allow the retrieval of detail about the port (to reflect its physical location): this permits drivers to correctly configure the port (for security keys for example).

5.4.3.3 Enumerate IFOE instances

Available to PF driver: Yes
Available to VF driver: Yes

This allows the driver to enumerate the number of IFOE subsystems in the system.

5.4.3.4 Get IFOE detail

Available to PF driver: Yes
Available to VF driver: Yes

This allows the driver to determine configuration details for the IFOE subsystems. This includes whether or not security is to be enabled on the XRSEC hardware and hence whether or not keys have to be programmed.

5.4.3.5 Assign security key

Available to PF driver: Yes
Available to VF driver: Yes

This allows the driver to set one of the two security keys for a specific IFOE subsystem. It does not enable or disable the key.

5.4.3.6 Enable security key

Available to PF driver: Yes
Available to VF driver: Yes

This allows the driver to enable or disable one of the security keys for a specific IFOE subsystem.

5.4.3.7 Enumerate ports

Available to PF driver: Yes

Available to VF driver: No

This allows the driver to determine the number of network ports present in the system and their handles.

5.4.3.8 Get port detail

Available to PF driver: Yes

Available to VF driver: No

This allows the driver to determine the details of a specific port. The information includes:

- MAC address
- Physical location
- IFOE subsystem

5.4.3.9 Get link statistics

Available to PF driver: Yes

Available to VF driver: No

This allows the driver to retrieve network port statistics for the specific port.

5.4.3.10 Ethernet packet transmit and receive

Available to PF driver: Yes

Available to VF driver: No

Packet reception and transmission is a low bandwidth operation to support heartbeats (indicating availability of MI450s) and LLDP (indicating the configuration of the network ports). The transfers are driven through conventional descriptor rings for reception and for transmission. The packet headers include the port handle from which or to which the packet was addressed.

5.5 Out of band connectivity

TODO (Still under development)

Out of band connectivity is required for both telemetry and for security. The connection is through a system MP which is reached over the SMN. For future expandability and to minimise maintenance, protocols should be based on agreed existing standards.

The handling of such data is problematic with existing DMTF protocols like NC-SI, PLDM monitoring and control and PLDM RDE (Redfish). This is because of the large number of network ports (NC-SI has a basic addressing range of 31 ports and the standard design for network NICs using PLDM monitoring is 10 ports).

5.6 IFOE

IFOE provides reliable transport of messages across Ethernet. Communicating endpoints transport messages using at least one pair of streams in each direction (one request, the other response). The endpoints may be connected by multiple streams using different network paths. Most of the IFOE functionality is handled by the hardware but firmware is required for various aspects of its behaviour which are discussed below.

5.6.1 IFOE

TODO (Still under development)

After reset, each IFOE block requires a number of initialisation steps. After (ECC) RAM initialisation, the

accelerator IDs, (network) port configuration, packet format need to be programmed. Additionally, the streams associated with each network path need to be configured.

When the system is configured to support failover, that is more than one physical port is available on the IFoE instance, firmware monitors the retry limits associated with message sending. If the retry limit is reached for a stream, the stream is directed to another active network port. This entails rewriting configuration information regarding the network port with which the stream is associated so that the system uses an alternative network port. Doing so necessarily reduces capacity on the alternative network port. The IFoE protocol is designed such that IFoE stream identification does not include the network port, allowing efficient switching.

There are several considerations here. First there's a limited time available to switch a stream since, by the time that failure is reported to the MP_IFoE, the IFoE block will have "used up" the available retries. Furthermore, in switching to the new stream, there's no guarantee of "instant service" so any transactions may take some time to emerge (although that isn't a problem for the MP_IFoE). If there is a failure, it is likely that more than one stream fails. The simplest strategy may be to switch streams as they hit their transport timeout limits¹⁴. In a system where more than one port is available for failover, the MP_IFoE round-robins between the available ports to spread the work more evenly across them.

A network port failure is more serious as all streams on the port are removed. It may be that it is infeasible to switch all of them to a new port in the time available.

IFoE failover needs to distinguish between a temporary failure (like a network port link drop) and receipt of a request with a bad accelerator ID. For the case of a bad accelerator ID, failover is not attempted and the IFoE subsystem goes into isolation (reporting errors as usual).

5.6.1.1 IFoE isolation

The IFoE link is considered to be the set of IFoE subsystems in the MI450 (across all MIDs). Essentially a link fails when an IFoE stream fails, and this might be for several reasons including:

- A remote node has failed
- All network ports on a subsystem have failed
- A network port on a subsystem and failover is not permitted

When the IFoE fails, the steps required for isolation are defined in the IFoE architecture document, section 7.3.10 (IFoE isolation) and 7.3.12 (UALINK ISOLATE message). Those steps require that the MP_IFoE:

- Disables the IFoE Tx scheduler.
- Places the inbound packet interface into Drop Mode.
- Disables SDP message generation.
- Places the inbound SDP interfaces into Drop Mode.
- Once the IFoE block enters the link-down state, sends a *UALink ISOLATE message* is sent from IFoE to DXS in the GPU: this message is triggered through the IFoE's CSR interface.

TODO messages need to be sent by IFoE to indicate a failure and to whom. (The message should include a reasonable amount of detail.)

5.6.1.2 IFoE link reset

To recover a link requires re-initialisation which is defined in the IFoE architecture document, section 12.2 (IFoE reinitialization): note particularly section 12.2.1 (Pod reinitialisation). Also note that there is no requirement to reset the IFoE systems themselves (unless the fault is so severe as to require that).

¹⁴ It isn't clear what support the IFoE block provides for multi-stream switching.

IFoE reinitialisation is performed to resynchronise the IFoE transport following an isolation event, and before restarting traffic. Note that the reinitialisation flow is distinct from the fatal error handling flow. Reinitialisation requires that the MP_IFoE:

- Quiesces and disconnects the SDP path between DXS and the IFoE block
- Waits for IFoE and link peer to both disconnect completer port.
- Quiesces and disconnects the SDP path between DXM and IFoE. A time out here causes a fatal error to be reported.
- Commands IFoE to disconnect the originator port.
- Quiesces the PFC, ensuring that all credits are returned for each active channel. A time out here causes a fatal error to be reported.
- Reconfigures XRPFC so that IFoE PCPs are mapped to the discard queue.
- Resets and reinitialises IFoE
- If encryption is enabled, resets the sequence numbers in the XRSEC

Following a fault which causes at least one IFoE station to require reinitialization, all other IFoE stations on the same network planes (within the same virtual pod) must be reinitialized. Restarting IFoE traffic requires coordination between nodes and so is orchestrated by the AIFM.

TODO What messages are sent to IFoE when the software has determined that things be recovered.

5.6.1.3 SDP disconnect/reconnect

The IFoE stack needs to support the SDP disconnect/reconnect protocols for power saving.

5.6.1.4 Power saving¹⁵

The underlying Ethernet ports use a considerable amount of power when idle (both receivers and transmitters). There is an opportunity for power saving by turning off the receivers on a proportion of the Ethernet ports when conditions permit (if the transmitters were powered down, the links would need to be retrained with the switch). Effectively this operates as a forced failover but, additionally, the underlying PHY has its receivers switched off. Recovering from the power saving requires that the PHY restore various old coefficients and then proceed autonomously with port recovery. The process needs to be fast to allow for the GPU workload's behaviour.

5.6.2 Security

IFoE links may be protected by encryption at the cost of extra latency. When disabled, the IFoE system uses a 64bit CRC for an additional, strong, integrity check. When enabled, the IFoE streams are protected by unique keys per stream using key derivation functions based on master keys that are assigned to either the host or guest (allowing security separation between the host and guest).

Although the hardware supports other encryption methods than IFoE, these are not available in the IFoE system of the MI450.

5.6.3 Priority flow control

Priority flow control splits traffic on a network port into different queues so that IFoE can take advantage of the Ethernet PFC (IEEE 802.1Qbb) properties. Allowing almost lossless packet forwarding across Ethernet and providing (at least) two priority channels with independent flow control. These properties support IFoE's requirement for a request and a response stream to another device.

Other than retrieval of telemetry, the MP_IFoE's role in managing PFC consists of configuring the priorities for the available network ports and queues, and setting the weighting of those queues.

¹⁵ Full implementation details are still being designed

5.7 Ethernet driver

The MP_IFoE supports the sending and receiving of Ethernet packets out-of-IFoE-band. These take place through the MR-PIO block within the Ethernet switch (EX). These packets are intended to be (very) low bandwidth both to and from the host. Since the Ethernet links are “local”, there should be limited multi-cast traffic and the packets that are sent and received can be controlled through the system design. Some simple MAC address filtering is provided to reduce transfers to the host based on the port’s MAC address and on multicast addresses. Out-of-IFoE-band packets must contain a VLAN tag so that they can be directed to MR-PIO.

5.8 Ethernet

The Ethernet links are provided through MACs and PHYs in each IFoE block. The actual IP involved for each of those is a development of existing IP. This allows pre-existing firmware drivers already developed¹⁶ to be ported for use on the MP_IFoE but with suitable adaptations for the system constraints.

5.8.1 MAC

The MAC control is relatively simple, providing APIs for retrieving statistics (in support of telemetry) and providing management of the MAC in terms of configuration, enabling and disabling. In the IFoE subsystem, the MAC also has responsibility for managing the link level retry functionality. Theoretically that exists “above” the MAC layer in the ISO network model but it is most easily considered as part of it. Link level retry requires both a compliant link partner (the switch) as well as enabling since it is an optional protocol. Link level retry is intended to handle soft errors and improve the local reliability of Ethernet connections. It does not impact the mechanisms for retry built into IFoE itself which operate in an end-to-end mode.

An important part of the MAC’s role is to pass link state information to other, interested, parties. This is important in the context of IFoE because the IFoE handling needs to act rapidly when a link goes down in a multi-port enabled block (so that streams can be switched to any available failover ports).

5.8.2 PHY

The PHY is concerned with managing the network link and comprises the PCS, PMA, PMD/MDI and AN/LT components. A state machine drives the link status which is driven by interrupts received from those components. The state machine uses the configuration information with relation to the Ethernet ports to establish the link, with underlying drivers handling things like auto-negotiation and link-training. The link training at high speeds can be protracted (over three seconds). The link status feeds into the MAC component.

5.9 Interprocessor communications

As noted in 4.9, the MP_IFoE communicates with other processors. The scheme adopted uses the same mechanism as already exists between other processors in the system, this uses three C2P mailboxes in each processor as follows:

Register	Description
MPxxx_CSPMSG_X	Command/response. Interrupt enabled.
[31]	DATA_READY flag
	- Requester clears when command ready
	- Responder sets when ready for communication
[30]	USE_SRAM flag, indicates data present in SRAM
[29:16]	Reserved

¹⁶ On KeystoneB (SmartNIC)

	[15:8] Command ID, set by requester, responder leaves intact
	[7:0] Status. Requester sets to 0, Responder sets to response status
MPxxx_CSPMSG_Y	Parameter #0
MPxxx_CSPMSG_Z	Parameter #1

Table 14: Interprocessor mailboxes

The USE_SRAM flag is only necessary where a command, or response, cannot be accommodated within the three mailboxes. Where it is necessary, the SRAM area must have been provisioned in the relevant processor with appropriate trust levels and access restrictions.

The following subsections describe the commands and their responses between the various processors.

5.9.1 ASP

Mailbox	ASP	MPIFoE
Command (interrupt)	149	16
Parameter#0	150	64
Parameter#1	151	65

Table 15: ASP and MP_IFoE mailboxes

MP_IFoE reserves a block of its SRAM for communications with the ASP of size 2K, which is split into 1K for ASP and 1K for MP_IFoE requests.

Command	Value	Description
TDISP_TRANSITION_READY	1	Ask MPIFoE if it is prepared to transition the TDISP state machine. If failure returned ASP should not transition the TDI. This must be done before sending the TDISP_TRANSITION_DONE command.
TDISP_TRANSITION_DONE	2	Inform MPIFoE that there was a state transition in the IFOE TDI. This is sent after the READY command is sent. This means that the nBIF registers are updated.
ASK_TDISP_REPORT	3	ASP requests TDISP Report from MPIFoE, which it returns through shared memory.
ACCELERATOR_ID	4	Send Accelerator ID to MPIFoE. The GPU PF driver sends the accelerator ID which DSM sends to MPIFoE.
IFOE_STATUS	5	Not used (may be removed).
GPU_ERROR	6	Inform MPIFoE that GPU TDI TDISP state has moved to ERROR.
GET_VF_BDF	7	Request VF BDF from MPIFoE.
RAS_ERROR_CONTROL	8	Enable or disable errors on a component
RAS_ERROR_INJECT	9	Inject an error
RAS_GET_ERROR_MASK	10	Report active mask on a component
RAS_ERROR_CLEAN	11	Clean up after injecting an error
SET_DRAM_MEMORY	12	Provide details of workspace buffer held in GPU's HBM memory
GET_VERSION	13	Return IFOE version

SET_MCA_TARGET	14	Set MCA target address
SET_ACTIVE_ACCELERATORS_MASK	15	Set bitmask indicating, for each destination accelerator ID, whether the accelerator exists and is active in the Cluster.
SET_LOCAL_ACCELERATORS_MASK	16	Set bitmask indicating, for each destination accelerator ID whether the accelerator is local to the Pod or not
GET_SRAM_MEMORY	17	Get details of shared MP_IFoE SRAM

Table 16: ASP commands to MP_IFoE

Command	Value	Description
TDISP_ERROR	1	Inform ASP that something has occurred which requires ASP to transition the IFOE TDI to ERROR.
GET_APCB_VALUE	2	Get value of requested APCB token
SEND_RID_INFO	3	Send IFOE RID to ASP

Table 17: MP_IFoE commands to ASP

The following subsections describe the detail of each command

5.9.1.1 ASP - TDISP_TRANSITION_READY

5.9.1.2 ASP - TDISP_TRANSITION_DONE

5.9.1.3 ASP - ASK_TDISP_REPORT

5.9.1.4 ASP - ACCELERATOR_ID

Identifies the GPU accelerator ID of the GPU connected to this IFOE system, in the range 0...1023, and the Virtual Pod ID of the GPU pod. This must be configured consistently between data fabric and IFOE.

Parameter	Bit	Width	Item	Description
0	0	32	ACCELERATOR_ID	Accelerator ID
1	0	32	VPOD_ID	Virtual POD ID

Table 18: ACCELERATOR_ID parameters

5.9.1.5 ASP - IFOE_STATUS

5.9.1.6 ASP - GPU_ERROR

5.9.1.7 ASP - GET_VF_BDF

5.9.1.8 ASP - RAS_ERROR_CONTROL

The ASP on the relevant MID (which is why the subsystem index is restricted to 18 distinct values) sends this command to amend the RAS error handling. Unconfigured subsystems are not affected by this command when the “all subsystems” index is passed. It is an error to address an unconfigured subsystem. See Table 7 for details of component identifier. See Table 9, Table 10, Table 11, Table 12 and Table 13 for element identifiers.

Parameter	Bit	Width	Item	Description
0	0	8	IFOESS	Subsystem index (0 ... 17). A value of 0xff indicates that all configured subsystems are affected.
	8	4	COMPONENT	Component identifier
	12	2	OP	Operation to perform: 0: Set mask absolute 1: Set mask relative (add to existing mask). 2: Clear mask relative (clear from existing mask).
1	0	32	MASK	Bit mask of elements to set or clear.

Table 19: RAS_ERROR_CONTROL parameters

Error responses:

- Invalid subsystem
- Invalid component
- Invalid operation

5.9.1.9 ASP - RAS_ERROR_INJECT

The ASP on the relevant MID sends this command to inject an error on a specific IFOE subsystem, component and element.

Parameter	Bit	Width	Item	Description
0	0	8	IFOESS	Subsystem index (0 ... 17).
	8	4	COMPONENT	Component identifier
	12	8	ELEMENT	Element identifier
1	0	32	FLAGS	Injection specific flags 0: SECDDED if clear inject single bit error, if set inject a double bit error

Table 20: RAS_ERROR_INJECT parameters

Error responses:

- Invalid subsystem
- Invalid component
- Invalid element for component

5.9.1.10 ASP - RAS_GET_ERROR_MASK

The ASP on the relevant MID sends this command to retrieve the active RAS mask for the specified subsystem and component.

Parameter	Bit	Width	Item	Description
0	0	8	IFOESS	Subsystem index (0 ... 17).
	8	4	COMPONENT	Component identifier
1	0	32	RSVD	Unused

Table 21: RAS_GET_ERROR_MASK parameters

Error responses:

- Invalid subsystem
- Invalid component

If no addressing errors, returns the active RAS mask for the specified subsystem's component in parameter#0.

5.9.1.11 ASP - RAS_ERROR_CLEAN

The ASP on the relevant MID sends this command following an error injection to perform any post injection cleanup on the specified IFOE subsystem, component and element. This only applies to the most recent RAS_ERROR_INJECT command and it is an error to alter the parameters that were presented to the previous injection command.

Parameter	Bit	Width	Item	Description
0	0	8	IFOESS	Subsystem index (0 ... 17).
	8	4	COMPONENT	Component identifier
	12	8	ELEMENT	Element identifier
1	0	32	FLAGS	Injection specific flags

Table 22: RAS_ERROR_CLEAN parameters

Error responses:

- Invalid subsystem
- Invalid component
- Invalid element for component
- Invalid parameter set

5.9.1.12 ASP – SET_DRAM_MEMORY

Parameters are passed in memory

Parameter	Bit	Width	Item	Description
0	0	32	BASE_LO	32MB base region, low word
1	0	32	BASE_HI	32MB base region, high word
2	0	32	SIZE	Size of region

Table 23: SET_DRAM_MEMORY parameters

5.9.1.13 ASP – GET_VERSION

Return the MP_IFoE firmware version: this command has no parameters.

Parameter	Bit	Width	Item	Description
0	0	8	TWEAK	MP_IFoE firmware version
	8	8	PATCH	
	16	8	MINOR	
	24	8	MAJOR	

Table 24: SET_DRAM_MEMORY response

5.9.1.14 ASP – SET_MCA_TARGET

Sets the MCA target address which is required by the hardware for proper MCA operation.

Parameter	Bit	Width	Item	Description
0	0	8	DIE_ID	Die ID
	8	8	SOCKET_ID	Socket ID

Table 25: SET_MCA_TARGET parameters**5.9.1.15 ASP – SET_ACTIVE_ACCELERATORS_MASK**

Sets the bitmask indicating, for each destination accelerator ID, whether the accelerator exists and is active in the Cluster. MP_IFoE uses this to configure the RX stream IDs.

Parameter	Bit	Width	Item	Description
0	0	32	ADDRESS	Address of 1024 bit accelerator mask in shared memory

Table 26: SET_ACTIVE_ACCELERATORS_MASK parameters**5.9.1.16 ASP – SET_LOCAL_ACCELERATORS_MASK**

Sets the bitmask indicating, for each destination accelerator ID whether the accelerator is local to the Pod or not. MP_IFoE uses this to configure SDP meta-data on the receive path.+

Parameter	Bit	Width	Item	Description
0	0	32	ADDRESS	Address of 1024 bit accelerator mask in shared memory

Table 27: SET_LOCAL_ACCELERATORS_MASK parameters**5.9.1.17 ASP – GET_SRAM_MEMORY**

Retrieves the details of the shared SRAM buffer for large requests and responses between the ASP and the MP_IFoE.

Parameter	Bit	Width	Item	Description
0	0	32	ADDRESS	Address of shared buffer
1	0	16	REQUEST_SIZE	Size of shared buffer for ASP
	16	16	RESPONSE_SIZE	Size of shared buffer for MP_IFoE. Follows immediately after ASP's buffer.

Table 28: GET_SRAM_MEMORY response**5.9.2 PM**

Mailbox	PM	MPIfFoE
Command (interrupt)	1	17
Parameter#0	2	66
Parameter#1	3	67

Table 29: PM and MP_IFoE mailboxes

MP_IFoE reserves a block of its SRAM for communications with the PM of size 2K, which is split into 1K for ASP and 1K for MP_IFoE requests.

Command	Value	Description

Table 30: PM commands to MP_IFoE

Command	Value	Description

Table 31: MP_IFoE commands to PM

5.9.3 MPIO

Mailbox	MPIO	MPIFoE
Command (interrupt)	1	18
Parameter#0	2	68
Parameter#1	3	69

Table 32: MPIO and MP_IFoE mailboxes

MP_IFoE reserves a block of its SRAM for communications with the MPIO of size 2K, which is split into 1K for ASP and 1K for MP_IFoE requests.

Command	Value	Description

Table 33: MPIO commands to MP_IFoE

Command	Value	Description

Table 34: MP_IFoE commands to MPIO

5.9.4 MP_IFoE

In a MI450 system there may be two MP_IFoEs, one on each MID. These need to communicate so that configuration, telemetry and other data can be coordinated.

Mailbox	MPIFoE primary	MPIFoE secondary
Command (interrupt)	24	25
Parameter#0	96	100
Parameter#1	97	101
Parameter#2	98	102
Parameter#3	99	103

Table 35: MP_IFoE mailboxes

6 Firmware architectural layering

This specification follows the layering model successfully used in other NTSG systems and is shown in Figure 10.

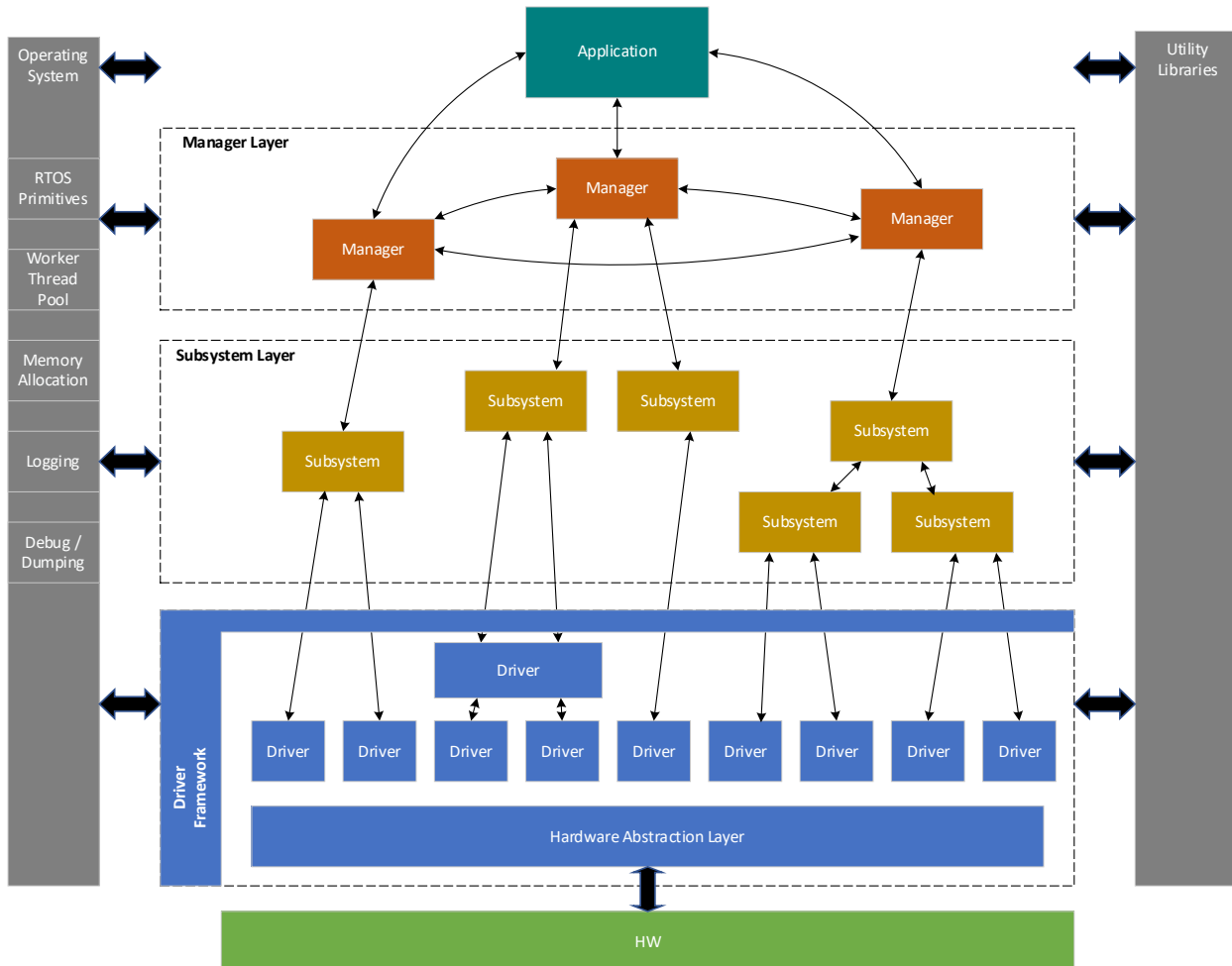


Figure 10: Firmware layers

6.1 Application layer

The application layer constitutes the top-level application that is responsible for initializing, starting and monitoring the lower layer components.

6.2 Manager layer

The manager layer contains software agents called managers, which implement specific portions of the firmware business logic. As such, they are tightly coupled to the use-case and will vary significantly between use-cases.

A manager is a firmware component with a clearly defined set of responsibilities. A manager usually takes exclusive ownership of one or more subsystems and is responsible for managing it throughout the lifecycle of the firmware. This involves handling subsystem initialization and teardown, allocation and deallocation of subsystem resources, coordination with other subsystems, performing operations on the subsystem(s) on behalf of other managers or host drivers, etc.

A manager will usually, but not necessarily, run as one or more independent threads of execution. In terms of distinction between managers and subsystems, it may be useful to think of it within the context of a classical operating system – drivers and subsystems are an equivalent of ‘kernel space’ of a classical OS – providing generic device and service abstractions – while the application and the managers are equivalent of ‘user space’ applications, implementing concrete system behaviour. Furthermore, one of the intents of the transition to the Zephyr RTOS and its MPU support is to create exactly such a line of separation, enforced by the MPU hardware. Drivers and subsystems would run in privileged (supervisor) mode with access to all memory, while managers would run in less privileged (user) mode, which provides memory isolation of both the supervisor mode and between managers themselves. Such a scheme allows for better containment of untrusted code (e.g. eBPF interpreters) and potentially easier recovery from misbehaving manager code.

6.3 Subsystem layer

A subsystem combines the functionality provided by device drivers and other subsystems to provide a coherent set of services to the upper layers. A subsystem is defined to be generic services (mainly hardware-backed) that do not include policy or business logic. The rationale for this is to allow entire subsystem modules to be used independently. Examples of subsystems include DMA services, Ethernet subsystem, persistent storage, etc.

A subsystem provides a specific high-level service by implementing the hardware-independent (common) logic of the service and orchestrating of one or more device drivers for the hardware dependent logic. Note that the subsystem layer provides services both to the layer above (the core service) and the layer below (common logic to the driver layer), i.e. the drivers can upcall into the subsystem to execute common functionality, coordinate with other drivers, or deliver data and events. A subsystem can also be layered on top of other subsystems to provide higher-level services, e.g. a structured storage service (partitions) over a raw storage subsystem.

6.4 Driver layer

The driver layer consists of hardware device drivers for the various IP blocks in the system. A driver is a module that interfaces directly to a hardware block, i.e. explicitly handles hardware resources - CSR registers, interrupts, etc. Its primary purpose is to provide an abstract view of the underlying hardware in such a way that (insofar as is possible):

- The underlying hardware can be modified, upgraded, or replaced with a similar functional block without significant disruption to the layers above
- Multiple hardware devices offering similar functionality (class of device), but different internal implementations can be used simultaneously in a design
- The underlying hardware and its associated driver can be re-used in other applications

These imply that any access to a hardware block from the upper layers must be mediated through a corresponding driver.

The scope of a driver should be limited to a coherent and self-contained set of functionalities that the underlying hardware block provides. In the case of simple hardware blocks, this will usually correspond to the complete functionality of the block. However, for complex, composite or multi-function blocks, it is recommended, where possible, to implement separate drivers for various distinct aspects of the block’s functionality. For example, an Ethernet IP core can consist of integrated MAC, PCS, Autonegotiation and PMA functions, but a separate driver should exist for each of the block’s functional layers. This promotes driver re-use in use-cases where only a subset of the block’s functionality is needed and also allows seamless migration to different hardware designs that utilize discrete IP blocks.

6.5 Hardware Adaptation Layer

Having a driver framework introduces a dependency on the framework in each of the drivers, limiting the potential use of the driver outside of the platform. Also, drivers may include a level of abstraction that potentially hides hardware functionality not relevant for the IFoE use case, further limiting usefulness of the driver to third parties implementing custom solutions.

A common solution to these problems is to split the driver into the HAL part, consisting of a relatively thin wrapper around the hardware block (mostly abstracting low-level register accesses and such) and a top-level driver which provides a higher level of abstraction (device class) and fits into the driver framework.

6.6 Operating system

Zephyr OS is a small footprint kernel designed for use on resource constrained and embedded systems. It supports many different CPU architectures and is well supported by the community. Zephyr OS provides a modular, full featured system on which to develop firmware.

7 Firmware components

The general arrangement of components within the system is shown in Figure 11. One of the intentions of the structure is that it makes unit testing of the various components much simpler. Unit testing must be an important consideration in design and development of modules.

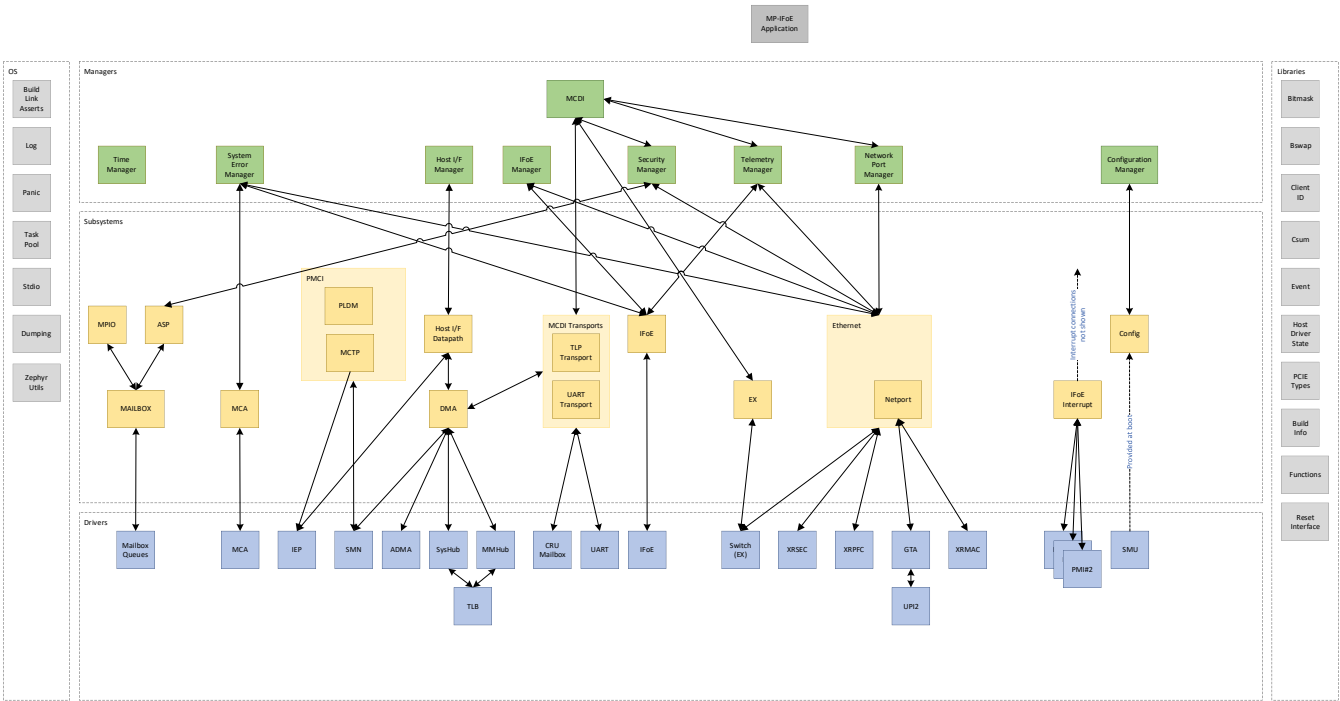


Figure 11: General arrangement of software components

7.1 Managers

7.1.1 Network ports

The network manager manages the network components of the I/OE subsystems. The manager draws a distinction between control plane activities and data plane activities. Control plane activities consist of managing the network port links, maintaining telemetry for the components, configuration of the SEC and PFC components. Data plane activities consist of low throughput transfers through the network ports.

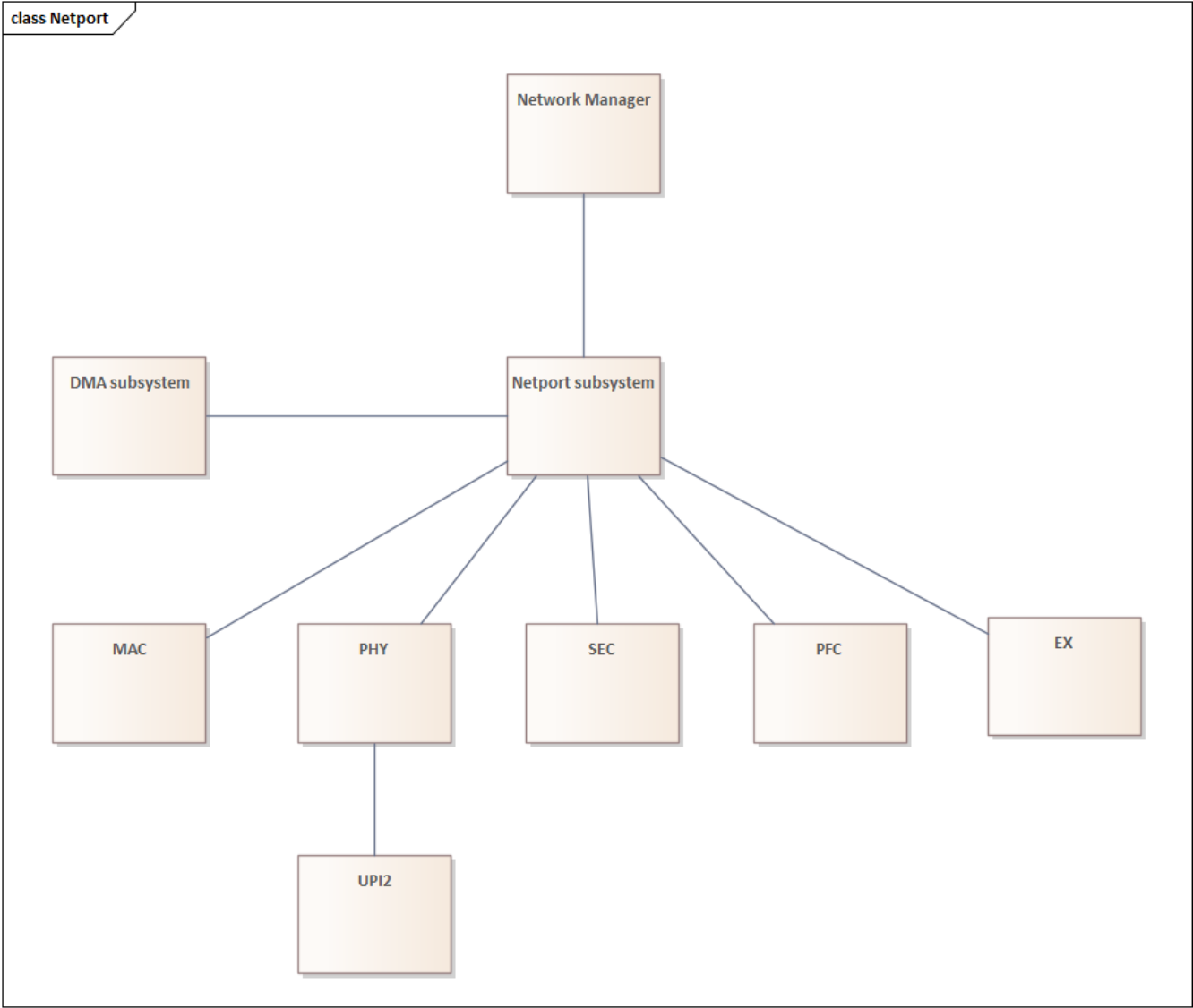


Figure 12: Network

7.1.2 IFOE

The IFOE manager is responsible for managing the IFOE subsystems. Other than initial configuration, the manager deals with network port link failures – failover. The IFOE manager also handles power reduction strategies when the IFOE links become idle.

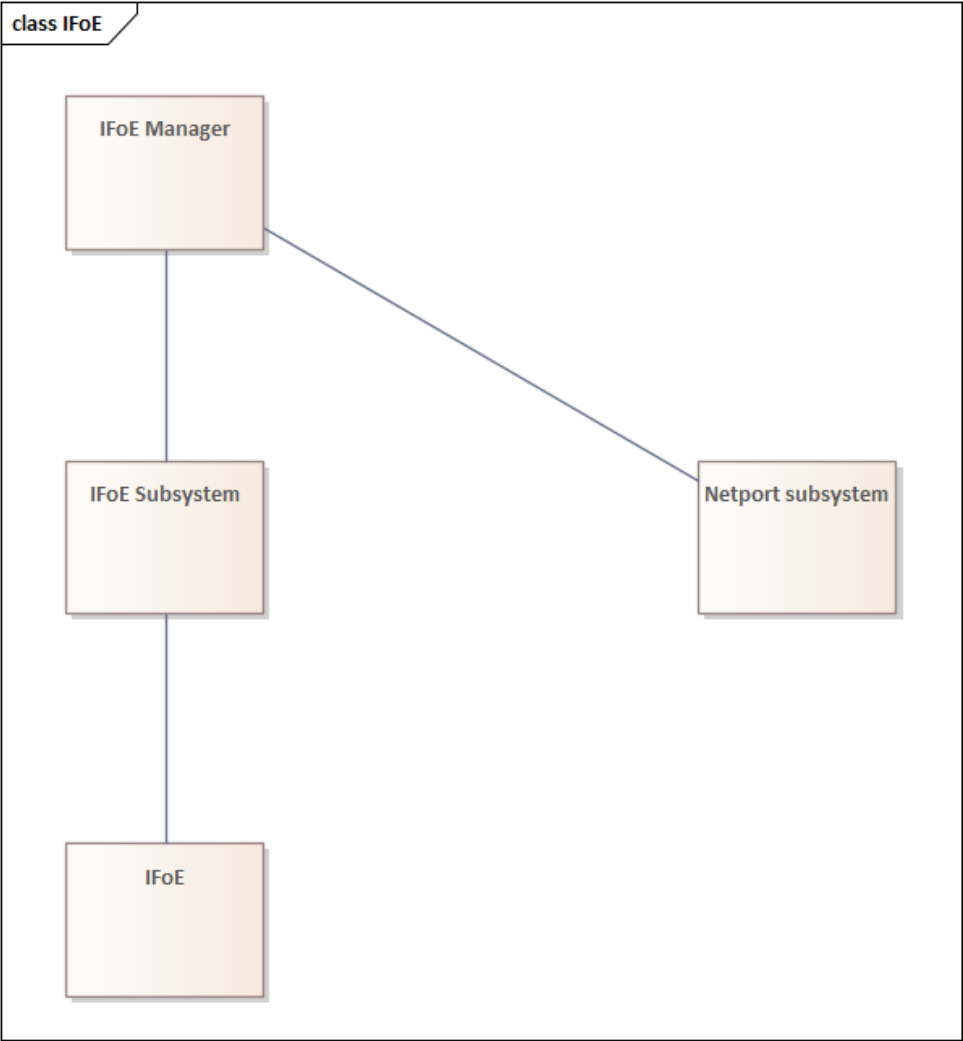


Figure 13: IFoE

7.1.3 Security

TODO (Security still being defined)

7.1.4 Telemetry

The telemetry manager provides a source of telemetry for the host and for any system entities needing such information. The principal sources of telemetry are from the IFoE subsystems and the network ports.

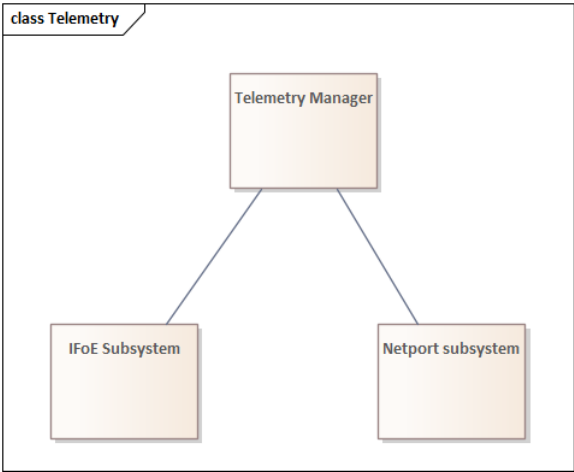


Figure 14: Telemetry

The telemetry manager interacts with the underlying hardware blocks through a standard telemetry driver. This avoids the need for the manager to know about every specific device in the system (SEC, MAC, etc.). Another advantage is that the telemetry manager can perform “proxied” DMA requests to obtain large chunks of data from the underlying blocks. This saves the driver having to handle DMA at all, unless required for its own operation.

The telemetry manager divides its share of the GPU_telemetry memory (each MP_IFoE gets an equal share) into blocks that are sized for the relevant driver’s statistics. Each set of blocks is preceded by a block that lists of statistics identifiers for that set. Blocks are rounded to the nearest power of two above 1k. A statistics block contains two generation values: one at the start and another at the end. When the two generation values are the same, this means that the statistics are valid. That allows other agents to read the data and to verify that they have a good data set.

This definition results in an arrangement as shown in Figure 15.



Figure 15: Telemetry set arrangement

The complete telemetry information contains sets for all of a subsystems components (IFoE, EX, XRSEC, XRPFC and XRMAC). The XRMAC statistics are arranged in blocks of ports (so all port 0s, all port 1s, etc.) – which makes it easy to handle the various network port modes. Additional sets for derived statuses such as network port link state, etc.

This arrangement of sets means that it is easy for the MP_IFoE to pass a block of statistics to any request agent, be that out-of-band or a driver as an address and length.

The telemetry manager collects the statistics through the (telemetry) driver’s API when it is notified that the statistics are to be collected, the general flow being described in 5.3.2. The telemetry manager performs the DMA on the driver’s behalf so that the driver does not need to do it itself. See 7.3.9 for details of the telemetry API.

7.1.5 Fault handling (MCA)

The error manager marshals errors arising from the components within the IFoE system and reports them

to the system controller through the MCA driver. This is because the IFOE subsystems don't integrate with the MCA architecture and the errors detected are effectively "soft". The error manager also notes when the DMA subsystem detects errors: typically, these arise as a result of data fabric being locked down because of errors.

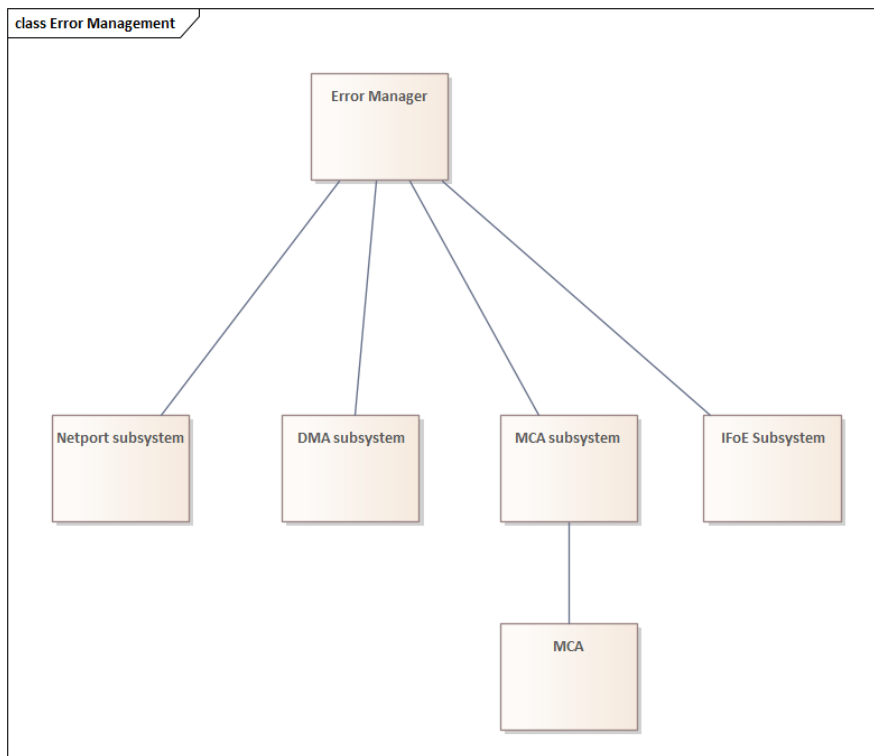


Figure 16: Error management

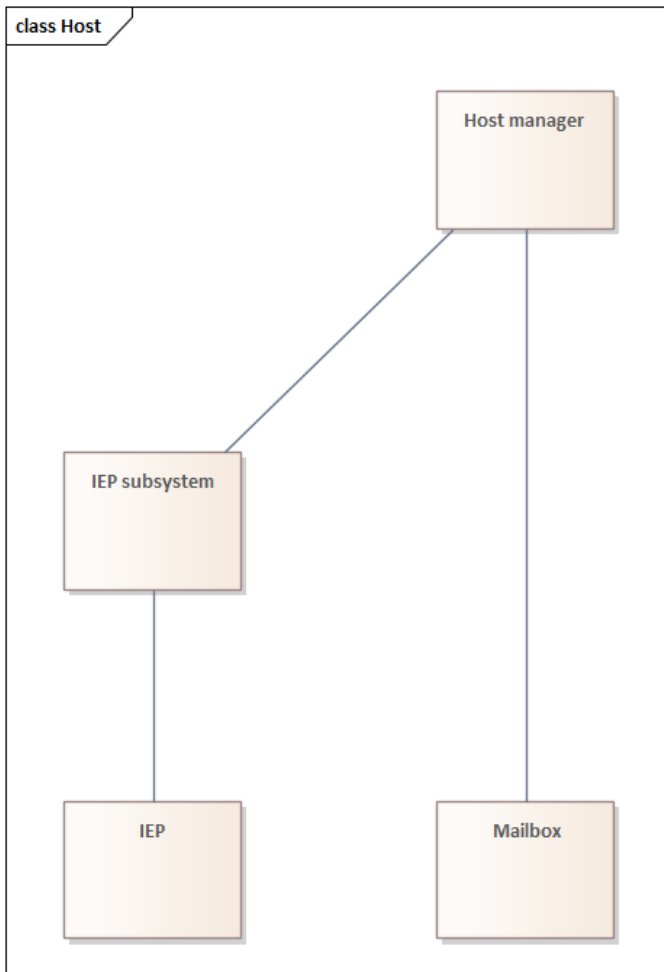
As with the telemetry manager, the error manager interacts with the underlying hardware blocks through a standard RAS driver. This allows the error manager to handle MCA errors in a uniform way irrespective of the block itself.

7.1.6 Configuration

The configuration manager extracts details from the platform topology so as to configure the IFOE subsystems correctly.

7.1.7 Host

The host manager provides host connectivity for the IFOE system. Only one IEP is active in the MI450 (in the primary MID) and these are filtered through the IEP subsystem. Requests are handled by forwarding to local (on the primary) and remote (on the secondary) MID as required. This provides uniform handling of all "PCIe" requests so far as each MID is concerned once the command distribution has been determined.

**Figure 17: Host**

7.2 Subsystems

Subsystems provide a coherent set of interfaces between managers and the drivers. Many of the subsystems may be relatively thin in terms of implementation. This isn't a disadvantage but allows for better design and easier unit testing of components.

7.2.1 DMA

The DMA subsystem provides clients with a means to perform data transfers without CPU intervention. The subsystem uses the ADMA driver to implement the transfers and assigns queues as follows:

1. SMN: SMN transfers to and from other devices.
2. Host: DMA transfers to and from host memory.
3. General: All other transfers.

The intention of separating the transfers in this way is to avoid the DMA channels competing for the same resource and to allow for the possibility of stopping transfers to a particular destination. A transfer may be multipart (to handle the case where multiple registers need to be read from an IP block for example). A message is posted to the client on completion.

The DMA subsystem provides allocation services for DMA blocks to avoid other drivers having to keep such details.

The DMA subsystem is also responsible for error handling occurring because of DMA transfers. The most likely failures are those associated with "sync flood" events where the data fabric is shut down and

transfers with the host (or GPU) fail. In such circumstances, the DMA subsystem will call back through a registered error handler. Clients will also be told about the failure of the transfer so that they may take appropriate action too.

The DMA subsystem does not manage any TLB mapping required for SYSHUB, MMHUB or SMN access: that is the responsibility of the client.

7.2.2 IFoE

The IFoE subsystem interfaces with the IFoE driver, providing a unified control interface for it.

7.2.3 Netport

The netport subsystem controls the drivers supporting the network functionality within the system. This does not include IFoE. The drivers provide data and control interfaces to the network as well as providing link information, etc. to clients. Combining all the EX, XRSEC, XRPFC and XRMAC functionality within one subsystem allows a uniform addressing of network ports within the system.

7.2.4 Tracelog

The tracelog subsystem provides a uniform method of all MPs within the MI450 to log data. Managers and subsystem may log to the common buffer (which is exposed externally) allowing the BMC to collate information about the system, where permitted by security and customer requirements.

7.3 Drivers

It is worth noting here that there are several distinct address spaces within the MI450:

1. MP_IFoE
2. SMN
3. GPU (aka Device)
4. Host

The last three are addressed through specific windowed regions (via TLBs) of the MP_IFoE address space and will have their own address type (C type) to avoid accidental mixing of addresses. A driver needs to obtain a suitable mapping in the MP_IFoE address space to access the relevant external address space. The intention is that as little dynamic allocation as possible will take place as this simplifies the driver design considerably.

Where possible, drivers should follow the semantics described by existing Zephyr drivers. Also note that some blocks present multiple drivers to separate functionality cleanly, these are typically those in the IFoE subsystems and relate to the block's actual functionality, telemetry and error handling.

7.3.1 IFoE RSMU

The RSMU driver is necessary for configuring and controlling Power Management Interrupts from the IFoE subsystems' RSMUs. Each RSMU contains a set of registers for configuring the PMI interrupts to an MP. Once configured, a PMI interrupt needs to be cleared in the RSMU before new interrupts from that line are permitted.

Although there is an RSMU driver in the amd-zephyr-common repository, this isn't very useful for IFoE because it maps and unmaps the RSMU for every SMN access: this is too inefficient for IFoE where there are potentially eighteen active subsystems all generating interrupts. Fortunately, all IFoE subsystem RSMUs exist in the same 1MB page. Therefore, a single driver that controls all RSMUs: this is reasonable because every RSMU is configured identically. The driver allocates an SMN page at start of day. An individual RSMU is identified by an index (0 ... 17) with a look up table for RSMU addresses. The API provided is similar to but

not identical with that of the existing (amd-zephyr-common) driver.

API

```
-- Enable the IRQ for rsmu_id
void ifoe_rsmu_enable_irq(unsigned int rsmu_id)

-- Disable the IRQ for rsmu_id
void ifoe_rsmu_disable_irq(unsigned int rsmu_id)

-- Clear the IRQ for rsmu_id
void ifoe_rsmu_clear_irq(unsigned int rsmu_id, uint32_t line_mask)

-- Get the interrupt status of the RSMU
uint32_t ifoe_rsmu_get_status(unsigned int rsmu)

-- Disable all PMI interrupts on all RSMUs
void ifoe_rsmu_disable_all_irqs(void)

-- Configure all RSMUs for the given priority with interrupts on given lines
-- This does not enable interrupts.
void ifoe_rsmu_config(uint32_t pmi_address)

-- Set the number of active RSMUs (from ifoe_ss subsystem)
void ifoe_rsmu_set_active_rsmus(uint32_t rsmu_mask)
```

7.3.2 IFoE PMI

Interrupts from peripherals arrive in “power management” FIFOs (see [SECIP] section 7.1.6 and [RSMU]). The FIFOs are held in SRAM and firmware configures them with a depth 1...32767 as required. An SMN connected device wishing to raise an interrupt writes to a mailbox register which adds an entry to the FIFO containing the source AXI address and raises an interrupt to the (local) CPU. Incoming interrupts have a priority associated with them that directs them to an appropriate FIFO. The IFoE subsystems aren’t configured for multiple priorities as interrupts are directed through the RSMU’s RAS interrupt rather than the three PMI interrupts. This means all interrupts for all subsystems come through a single interrupt.

The IFoE PMI driver allocates a single FIFO sized to allow individual interrupts from every IFoE subsystem (so at least 8 x 18 entries). Each FIFO will receive the instance ID of the IFoE subsystem, so the first job of the driver is to find the IFoE sub-system index. Appendix A: Initiator IDs shows that the instance ID of subsystems are disjointly numbered. Finding the subsystem index is most easily achieved by a binary search rather than any form of linear search given that there are potentially eighteen sub-systems.

Within an IFoE subsystem, each IP block connects with a different line and the PMI driver allows a different driver to be connected to each line. When the interrupt occurs the PMI driver calls the registered handler for the line with the user supplied context and the MP_IFoE base address of the IFoE subsystem (that is the mapped SMN address).

The IFoE PMI driver dispatches interrupts from a high priority thread rather than the interrupt itself because that avoids extended processing within the interrupt routine (since all interrupts from all subsystems come through a single vector). The queue servicing agent, operating on a high priority thread keeps reading from the PMI interrupt’s FIFO, processing the interrupts until the FIFO is empty. This avoids excessive interrupt latencies, keeps the interrupt ordering (by virtue of the FIFOs) and allows drivers to manage interrupts themselves (rather than having the handlers manage the driver interrupts directly). When the FIFO is empty, the PMI interrupt is re-enabled.

The IFoE PMI driver relies on the IFoE RSMU driver (7.3.1) and so loads after it.

API

```
-- IRQ handler
typedef void (*ifoe_pmi_client_t)(const void *ctx, mem_addr_t ss_base)
-- IRQ registration function
void ifoe_pmi_register_client(unsigned int line_id, ifoe_pmi_client_t client, const void *ctx)
```

7.3.3 IFOE line interrupts

As noted in 4.9, Zephyr's interrupt hierarchy is fully populated by the existing hardware leaving no space for additional hierarchies to be supported. The depth of the interrupt hierarchy for the IFOE subsystems is shown in Appendix B: IFOE sub-system interrupt hierarchy, from the perspective of registers to read to get the detail of a specific interrupt for an IP block.

The depth of the platform's interrupt hierarchy, the number of IFOE sub-systems and the depth of the IFOE interrupt hierarchy means that it is infeasible to run all the code in an interrupt routine without extended interrupt latencies. To this end the following scheme is deployed which makes use of the fact that PMI interrupts, through which the IFOE subsystems notify the MP_IFOE of events, are stored in FIFOs within the the MP_IFOE's SRAM.

Each interrupt line (that is component within an IFOE subsystem) instances a separate IFOE line driver. This is specialised, as required, the IFOE subsystem. This allows the correct context to be passed to the registered interrupt handler. The driver models the semantics of the Zephyr interrupt framework except that there is no requirement to supply a context when registering the handler because the line driver derives that from a knowledge of the interrupt type (EX, XRMAC, etc) and the base address of the IFOE subsystem. Interrupt numbers are defined for each top level interrupt within the particular line driver. It is the responsibility of the driver (interrupt handler) to descend the remaining interrupt hierarchy.

Each (specialised) aggregator has its own table of interrupt entries sized appropriately to the number of supported interrupts (which avoids wasting SRAM with sparse tables). The aggregators are bound to the driver via the aggregator IDs and device tree entries. Only one level of interrupt is supported: relevant drivers handle all enabled interrupts internally.

It is important to note that the top level interrupts are always enabled and can only be disabled for the IFOE block within the IFOE subsystem.

The IFOE PMI driver relies on the IFOE PMI driver (7.3.2) and so loads after it.

API

```
-- IRQ handler
typedef void (*ifoe_irq_handler_t)(void *arg)
-- IRQ registration function
int ifoe_irq_connect(ifoe_rsmu_irq_t irq_id, ifoe_irq_handler_t handler)
```

7.3.4 MMHUB

The MMHUB driver provides access to device memory. It is concerned with mapping GPU addresses to the MP_IFOE's address space.

7.3.5 SYSHUB

The SYSHUB driver provides access to host memory together with the ability to signal interrupts to the host. It is concerned with mapping host addresses to the MP_IFOE's address space.

7.3.6 ADMA

The ADMA driver manages the ADMA hardware. This is a relatively simple driver exposing a number of channels that can each process one request at once. Each request may comprise a number of blocks with callbacks at the end of each block or at the end of the request.

The ADMA driver is likely not to follow the Zephyr model because that requires a double translation of addresses (source → Zephyr and → ADMA) which is likely to be inefficient. The ADMA descriptors are stored in a ring buffer allocated at start of day.

The ADMA driver is itself managed by the DMA subsystem and isn't used directly by other drivers or subsystems.

7.3.7 CRU/Mailbox

The CRU driver provides access to the mailboxes held within the CRU. Different components within the MI450 use mailboxes for communication.

7.3.8 SMN

The SMN driver provides access to the SMN. It is concerned with mapping SMN addresses to the MP_IFoE's address space.

7.3.9 Telemetry

The telemetry driver provides a uniform API for accessing the telemetry information in an IP block. The interface allows a set of data to be prefetched (by the telemetry manager) before the statistics are combined by the driver and presented back to the telemetry manager. Some data may be fetched by the driver itself (where the cost of DMA would be too high).

The telemetry driver provides an API of the following form:

Function	Description
prepare	Prepare to collect statistics. The driver takes any actions necessary for statistics collection (which might involve writing a capture/clear register).
get_descriptors	Get an array of (DMA) descriptors. This should be a static, but non constant set of descriptors. The telemetry manager updates the source addresses to reflect the correct IFOE instance address. The destination addresses are also amended to reflect the offset into scratch memory. The array may be empty if no DMA is required.
process	Process the collected statistics. The driver is passed the scratch area address and the address of the statistics block.
get_descriptor_ids	Gets an array of descriptor IDs that are present in the statistics block. The IDs are returned in the order in which they occur.

Table 36: Telemetry driver API

7.3.10 RAS

The RAS driver provides a uniform API for accessing error (MCA) information in a block and for providing error injection facilities.

7.3.11 IFOE container

The IFOE container driver is the parent of all the individual drivers in a (hardware) IFOE subsystem. Its role is to map the subsystem into the MP_IFoE's address space, using the SMN driver, so that the individual drivers don't need to be aware of SMN. This avoids individual drivers wasting time mapping memory in and out of MP_IFoE's address space – effectively the IFOE subsystems are permanently mapped into the MP_IFoE's address space. The IFOE container driver also provides numbering and availability services for the IFOE subsystem.

7.3.12 IFOE

The IFOE driver initialises, configures and monitors the IFOE block. The configuration and initialisation sequences are described in the IFOE MAS. The IFOE block is not operative until the configuration, which

includes pod identifiers and network port connections, are complete and the network links are up.

Also presents RAS and Telemetry drivers.

7.3.13 Switch (EX)

The EX driver is a simple Ethernet packet receive and transmit interface through the FIFOs present in the MRPIO component of the EX block. The driver configures the maximum MTU required. The EX hardware presents a simple FIFO based interface to receive (and transmit) packet data. In particular, there's no provision for dropping a partly read packet: all data must be read from the FIFO to advance to the next packet (including truncated packets that have exceeded the MTU). This requires careful design to avoid pathological cases where the EX driver reads data continually because of, say, a broadcast storm. The expectation, however, is that traffic will be very low (LLDP and heartbeat traffic).

Also note that the hardware interface requires additional headers to be added (transmit) or removed (receive) over and above the standard Ethernet packet. Packets, unless specifying an LLDP MAC address, will have a VLAN tag added: the EX driver will complete the VLAN details when transmitting packets, with the EX driver filling in the priority. VLAN tags are stripped on receive. Attempts to send a packet that contain a VLAN tag will fail. This behaviour is enforced so that a host driver cannot interfere with the IFOE packet flows.

Also presents RAS and Telemetry drivers.

7.3.14 SEC

The SEC driver is responsible for the XRSEC block in the IFOE subsystem. After a cold boot the driver must perform the CAVP tests to ensure the correct cryptographic operation of the block. The encryption/decryption functionality may be enabled/disabled dynamically (though obviously not while there are active traffic flows which will break the IFOE streams).

Also presents RAS and Telemetry drivers.

7.3.15 PFC

The PFC driver is responsible for the XRPFC block in the IFOE subsystem. The role of the PFC driver is more complicated than it might first appear. Not only does it manage the programming of the ports, queues and the weights associated with them in for priority flow control but also it programs IFOE related parameters.

The expected assignment of PCPs is as follows:

- 0: IFOE requests. This is important because it avoids the need for a VLAN tag which effectively wastes network bandwidth.
- 1: IFOE responses.
- 2: MRPIO traffic. This is local traffic to the IFOE system (heartbeats, etc). Ideally local traffic is transmitted with a suitable VLAN tag (this avoids additional programming in the XRPFC).
- 3: Discard. Traffic received here is simply dropped. This is necessary in some IFOE states as well as for unwanted traffic.

The XRPFC has two packet classifiers per port that can direct management (MRPIO) traffic. Only one of these classifiers needs to be programmed to support LLDP (which uses a MAC address of 01:80:C2:00:00:0E for a single physical link). The other packet classifier can be left unused (since "local" traffic can be directed by VLAN tags).

Also presents RAS and Telemetry drivers.

7.3.16 MAC

The MAC driver is responsible for the XRMAC block in the IFoE subsystem.

Also presents RAS and Telemetry drivers.

7.3.17 UPI2

The UPI2 driver provides the mechanism for the GTA driver to communicate with the processor that handles PMA and ANLT operations.

7.3.18 GTA

The GTA driver provides the PMA driver for a network port. The driver is complicated by the asynchronous nature of the underlying (UPI2) transport.

7.3.19 IEP

7.3.20 MCA

The MCA driver manages the access to the MCA block within the MP_IFoE tile. There are potential race conditions that need to be considered here as well as the use of the correctly sized accesses to the register block.

7.3.21 Tracelog

The tracelog driver is implemented through the common driver to be found in the amd-zephyr-common repository. It provides a standardised method of logging debug information to a circular buffer held in the MP_IFoE's distributed trace buffer (which is held in the DFD block).

7.3.22 Postcode

The postcode driver is implemented through the common driver to be found in the amd-zephyr-common repository. It provides a standardised method of providing POST codes to the system software.

8 Host and Driver

TODO (Still under development)

8.1 Host (PF)

The host driver provides an IFOE host driver for IFOE configuration. A driver for the Ethernet ports is provided, allowing management including link control, status and telemetry. The driver also supports low-bandwidth, raw Ethernet packet send and receive. Typically, this might send LLDP messages or heartbeat messages for link and GPU detection and verification.

8.2 Guest (VF)

The guest driver provides an IFOE guest driver which allows for IFOE tuning, status and telemetry.

9 Opens

Heartbeat handling.

System details:

- Bulk transfer of telemetry data.
- Time of day synchronization. Necessary to ensure that telemetry and error data are presented consistently across the system.
- Error injection

10 Appendix A: Initiator IDs

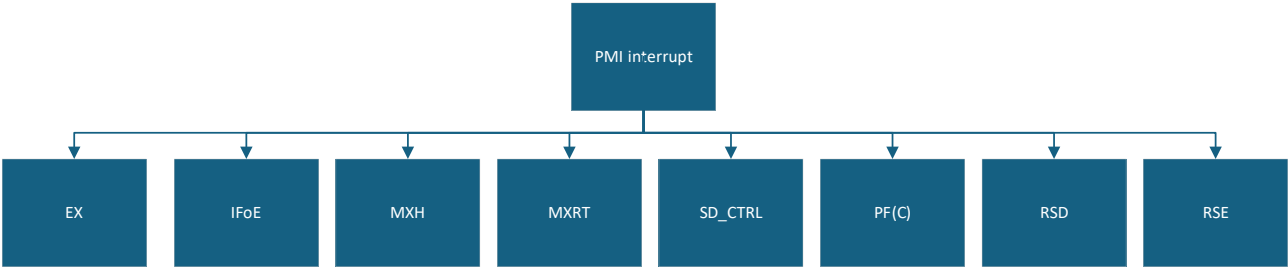
Table 37 shows the initiator IDs assigned to each IFoE subsystem on a MID.

IFoE-SS	Initiator ID
ifoe0	121
ifoe1	122
ifoe2	123
ifoe3	199
ifoe4	200
ifoe5	201
ifoe6	202
ifoe7	203
ifoe8	238
ifoe9	239
ifoe10	240
ifoe11	241
ifoe12	242
ifoe13	243
ifoe14	244
ifoe15	245
ifoe16	246
ifoe17	247

Table 37: IFoE Subsystem initiator values

11 Appendix B: IFoE sub-system interrupt hierarchy

The top-level interrupt hierarchy for an IFoE subsystem is shown in Figure 18. The following figures show interrupts associated with each block. MXH and MXRT represent the XRMAC, RSD and RSE represent the XRSEC, PF(C) the XRPFC and SD_CTRL the GTA PHYs.



Top level interrupt structure
One PMI interrupt sources eight interrupts per IFoE subsystem

In the following figures () indicates number of possible interrupts
Lines with numbers gives number of instances

Figure 18: Top level interrupt hierarchy

The interrupts are connected to the PMI inputs of the IFoE subsystem’s RSMU as shown in Table 38. Note that because of the way in which the PMI interrupts work, the implicit priority is the lowest bit. With the exception of IFoE, the top level interrupts are all, always enabled and cannot be disabled. Furthermore all sub interrupts are enabled by default. This **requires** that drivers disable interrupts unless they require them.

IP block	PMI interrupt pin	Comments
EX	0	Ethernet switch
IFoE	1	IFoE protocol
MXH	2	MAC (principally errors)
MXRT	3	MAC (TX, RX,
SD_CTRL	4	PHY control (UPI2)
PFC	5	Priority Flow Control
RSD	6	XRSEC (receive)
RSE	7	XRSEC (transmit)

Table 38: PMI interrupt pin allocation

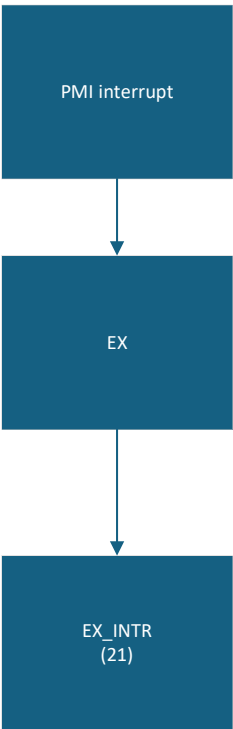


Figure 19: EX interrupt hierarchy

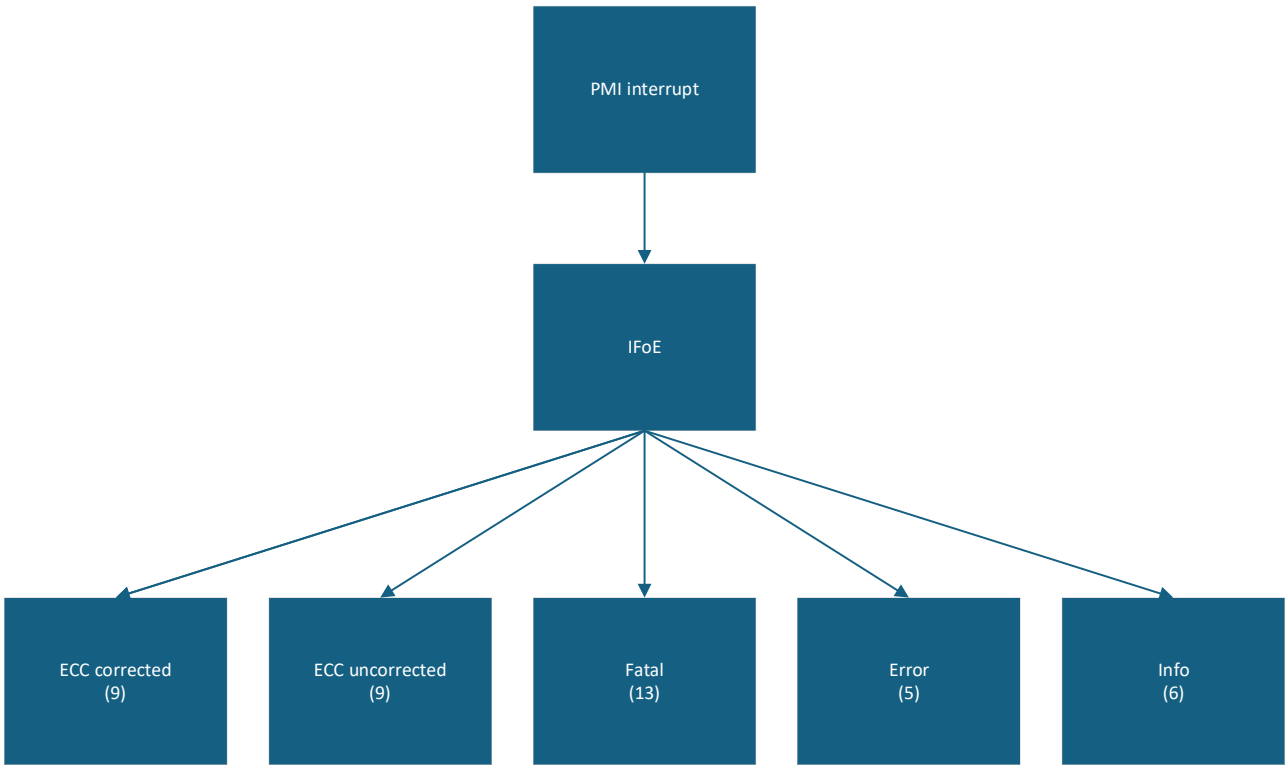


Figure 20: IFoE interrupt hierarchy

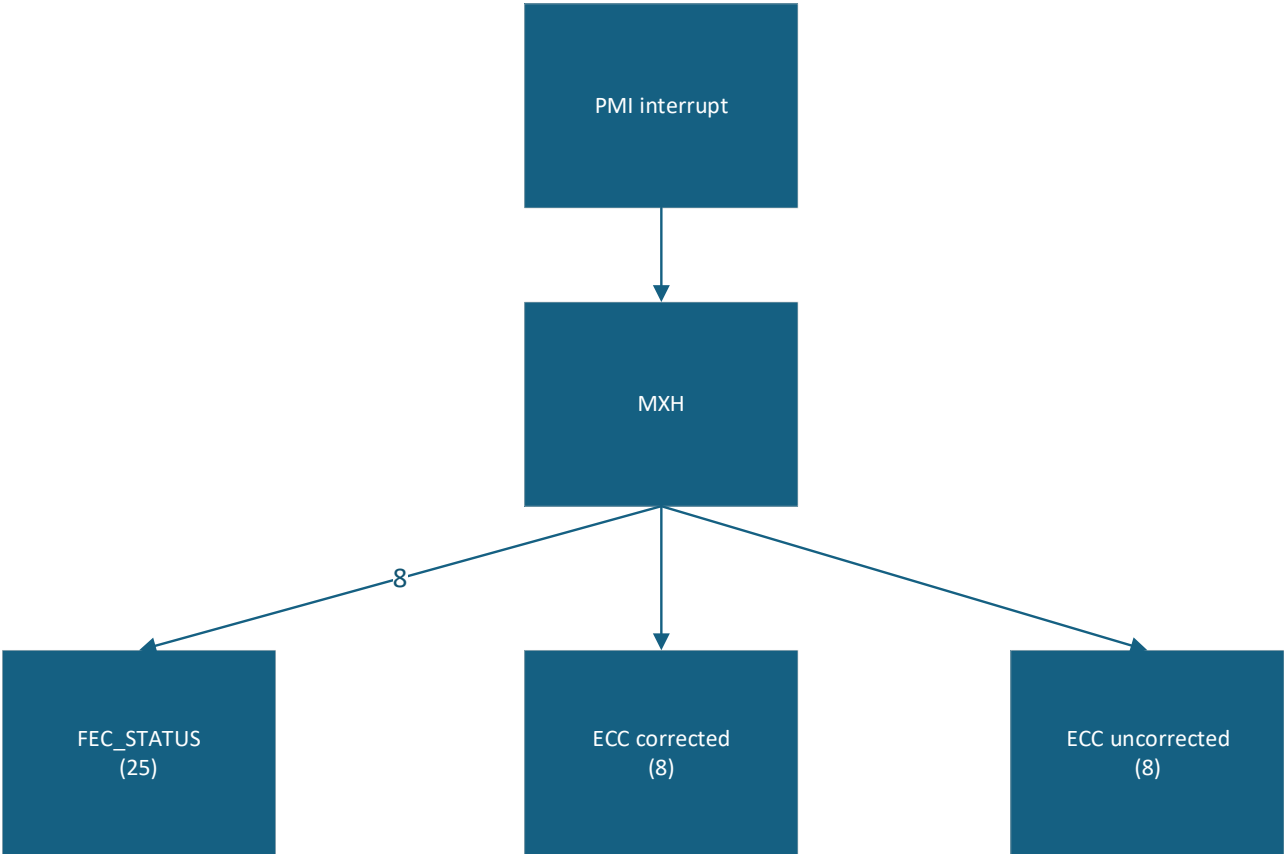


Figure 21: MXH interrupt hierarchy

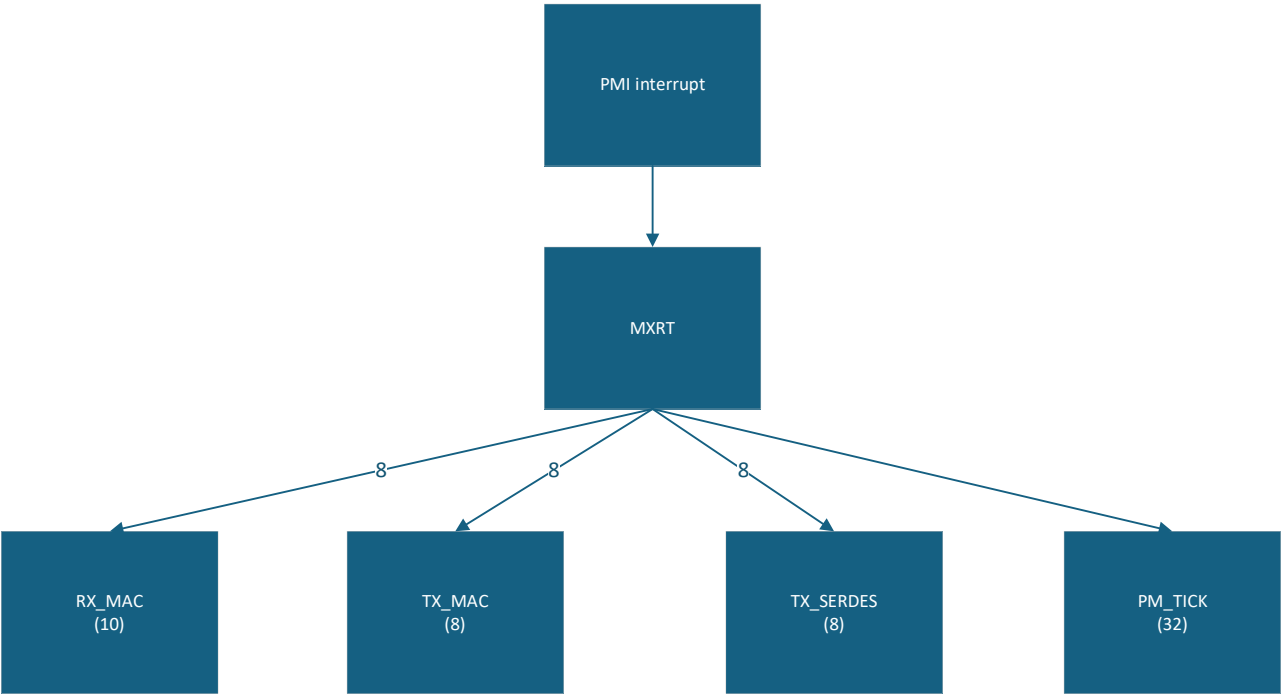


Figure 22: MXRT interrupt hierarchy

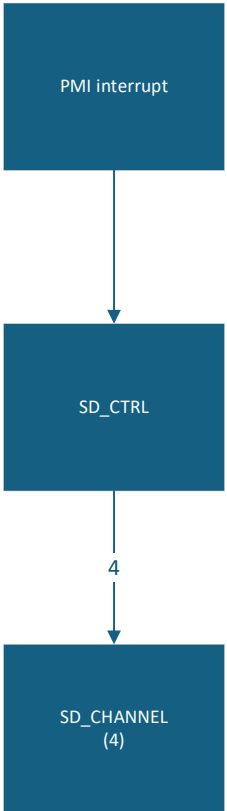


Figure 23: SD_CTRL interrupt hierarchy

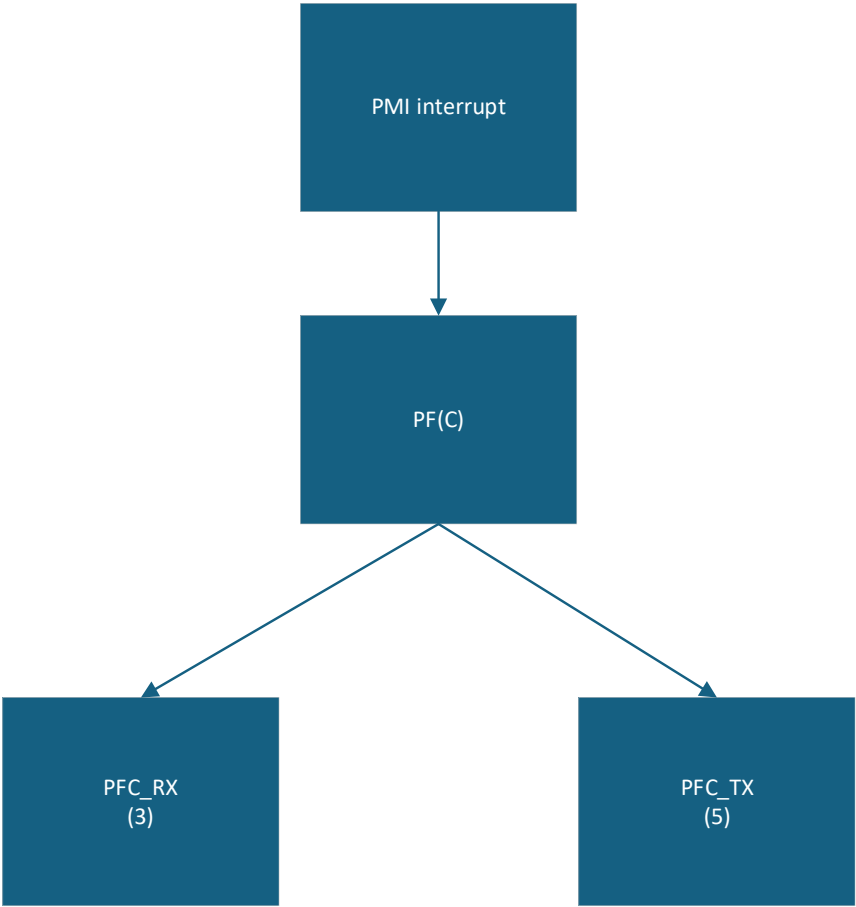


Figure 24: PFC interrupt hierarchy

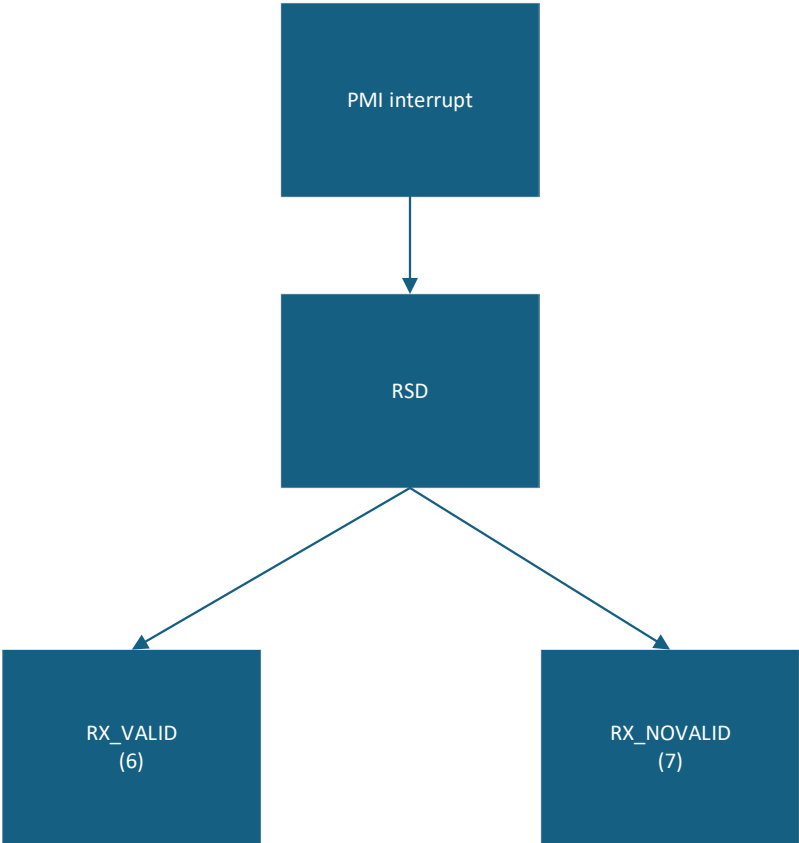


Figure 25: RSD interrupt hierarchy

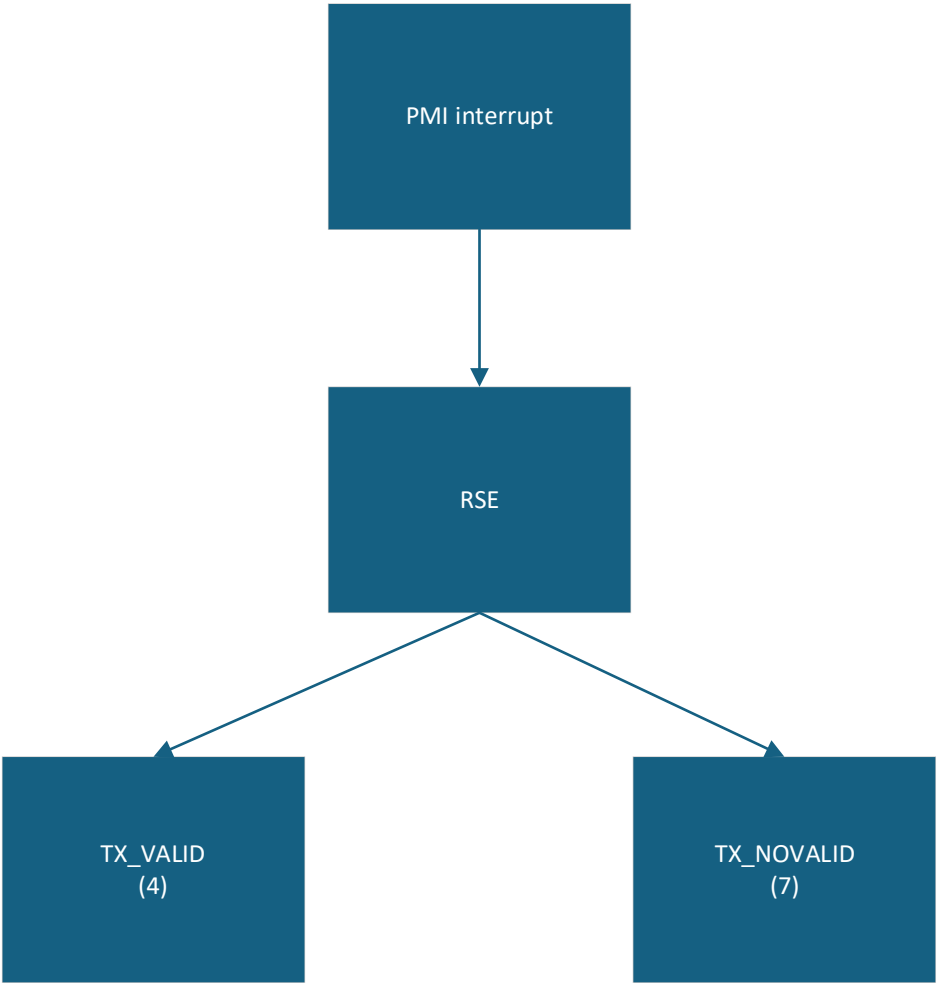


Figure 26: RSE interrupt hierarchy

12 Appendix C: Configuration items

This section considers the different configurable elements of IFoE and specifies the set of things configured by the various entities that interact with IFoE.

12.1 Configurator

This table lists the different entities that configure aspects of IFoE.

Configurator	Description
AIFM	The AIFM client runs in the host/hypervisor of each x86. It receives configuration from the centralized cluster controller and uses this to configure the IFoE datapath via the IFoE kernel driver.
VPM	Virtual Pod Manager. Client that runs in Guest OS and used to coordinate between tenants in a GPU Pod.
BIOS	Configuration via MII forms and stored in Flash and passed to MP-IFoE via mailbox communications with other MPs.
Firmware	Handled by firmware internally, either as a result of events such as a Network Link going down or derived from other configured information.
Constant	Configuration that is expected to be constant and can be configured to a fixed value in the IFoE firmware
Fuse	Configuration that is retrieved from the MI450's fuses

Table 39: Configurator types

12.2 Config Phases

The design of the IFOE datapath requires almost all configuration items to be set up before the datapath is used. To support this and additional requirements around the timing and ordering of certain aspects of the initialization, a sequence of configuration phases is defined.

Phase	Description	Firmware Actions
SYSTEM_CONFIG	After the MP-IFOE firmware boots it will enter this state waiting for the Network Configuration. This configuration is required early in the boot process, before the Host boots to allow IFOE to bring up the Network Ports. This configuration is stored in the BIOS as a set of APCB tokens and communicated to IFOE via the ASP processor.	• Configure the Network Port mode for the configured IFOE instances • Power on (staggered) the SERDES for the configured IFOE instances • Start the link training process on configured Network Ports • Read the MAC address pool from the fuses
PROVIDER_CONFIG	After the network configuration is complete, the firmware enters this state waiting for the IFOE datapath configuration. This configuration is provided from AIFM via the Host kernel driver i.e. after the Host OS has loaded.	• Disable and power off Network Ports of any IFOE instances that are not being used. • Configure IFOE datapath
TENANT_CONFIG	After the core IFOE datapath configuration is complete, the firmware enters this state waiting for the tenant configuration.	• Configure encryption, if being used • Enable set of remote accelerators happy to receive traffic from
MISSION_MODE	Configuration is complete, datapath enabled.	

Table 40: IFOE configuration phases

12.3 Fuse Configured

These configuration items are blown into fuses when the system is built.

Name	Parameters	Description	Scope	Config Phase
MAC address	(Base address, pool size)	Pool of 96 MAC consecutive addresses programmed into fuses during manufacture. These MAC addresses are for out-of-band (non-IFoE) network communications.	Global	SYSTEM_CONFIG

Table 41: Fused configuration items

12.4 BIOS Configured

These configuration items are stored in flash and communicated to the MP-IFoE during the boot process via interactions between MP-IFoE and the other MPs.

Name	Parameters	FW default	Description	Scope	Config Phase
IFoE Configured Stations	36-bit bitmask	0	Bitmask of the IFoE stations (instances) that are configured for use. There are 18 IFoE instances per MID and therefore up to 36 per MI450.	Global	SYSTEM_CONFIG
Network Port Mode	<1x800G 2x400G 4x200G>	4x200G	Enumeration of the Network Port Mode that IFoE should operate in. All instances of IFoE in all nodes in the cluster should be configured in the same mode.	Global	SYSTEM_CONFIG
Link-Layer Retry	<On Off>	Off	Boolean indicating if MAC Link-Layer Retry is configured. This configuration should be consistent between IFoE network ports and the connected Ethernet switch. It is assumed that this will be configured globally on or off across the cluster. ("should be" because LLR is a negotiated protocol between link layers.)	Global	SYSTEM_CONFIG

Table 42: BIOS configuration items

12.5 AI Fabric Manager (AIFM) Configured

These configuration items are passed from AIFM to MP-IFoE via the IFoE kernel driver. Note that some configuration is done via the PF and other configuration via the VF.

Name	Parameters	FW default	Description	Scope	Config path	Production use	Config Phase
Accelerator ID	(Accelerator ID, Virtual Pod ID)	0, 0 ¹⁷	Identifies the GPU accelerator ID of the GPU connected to this IFOE system, in the range 0...1023, and the Virtual Pod ID of the GPU pod. This must be configured consistently between data fabric and IFOE. To achieve this, the AIFM also configures the Data Fabric's accelerator ID through the GPU driver and whence through to the ASP.	Global	IFoE PF (& GPU driver) ASP ¹⁸	Yes	PROVIDER_CONFIG
IFoE Active Network Ports	36 x 4-bit bitmasks	0	Bitmasks of the IFOE Network Ports that will be used for each IFOE subsystem. This bitmask should be a subset of the Network Ports that have implicitly been enabled by the BIOS. The firmware will disable and power off all inactive ports for power saving.	Global	IFoE PF	Yes	PROVIDER_CONFIG
Active Accelerators	1024-bit bitmask	0	Bitmask indicating, for each destination accelerator ID, whether the accelerator exists and is active in the Cluster. Used to configure RX stream IDs.	Global	IFoE PF ASP	Yes	PROVIDER_CONFIG
Local Accelerators	1024-bit bitmask	0	Bitmask indicating, for each destination accelerator ID whether the accelerator is local to the Pod or not. Used to configure SDP meta-data on receive path.	Global	IFoE PF ASP	Yes	PROVIDER_CONFIG
Virtualization Mode	<BAREMETAL VIRTUALIZED>	VIRTUALIZED	Indicates whether IFOE is operating in a hypervisor-guest configuration or	Global	IFoE PF	Yes	PROVIDER_CONFIG

¹⁷ Likely to be required to be set before IFOE considered usable¹⁸ When ASP configured may need to disable all similar items that can be PF configured

MP IFOE ARCHITECTURE DRAFT 1K

			baremetal. This configuration controls which PCI Fn does stream→path spreading and must be configured the same way in all instances of all nodes in the cluster. Note that for the virtualized case, the guest must be able to query/validate the configuration via the VF.				
Encapsulation Type	<ETHERNET IP TF1>	ETHERNET	Set the encapsulation used for IFOE. This must be set consistently across the cluster- all nodes and all switches must be configured in the same way.	Global	IFoE PF	Yes	PROVIDER_CONFIG
IFoE Ether Type	16-bit integer	Ethernet & IP => 0x1F0E TF1 => 0x1F1E	The default value is 0x1F0E and must be set consistently across the cluster. A host interface is provided to set the ethertype but it is not expected that the driver would use this.	Global	IFoE PF	No	PROVIDER_CONFIG
IFoE IP Protocol Type	8-bit integer	TODO	This must be set consistently across the cluster. A host interface is provided to set the protocol type but it is not expected that the driver would use this.	Global	IFoE PF	No	PROVIDER_CONFIG
TTL Value	8-bit integer	64	The firmware will set this to a sensible default value. A host interface is provided to set the TTL but typically it is not expected that the driver would use this. Default value chosen is the IP default value.	Global	IFoE PF	No	PROVIDER_CONFIG
MTU	16-bit integer	4450	IFoE mission mode has a defined MTU size. A host interface is provided to set the MTU but it is expected the only reason to use this would be to	Per IFOE instance	IFoE PF	No	PROVIDER_CONFIG

experiment with host tuning.							
Loopback	(IFoE <On Off>, MAC <On Off>)	Off, Off	Configures loopback. This feature has no production use case but is used for pre-silicon testing and for silicon manufacturing test. Two different types of loopback are required: loopback within the same network port at the MAC and loopback by physically connecting together two network ports outside the chip. To support this, there are two separate controls for loopback: • IFoE loopback controls swapping of fields inside the IFoE datapath to ensure IFoE does not dropped looped back traffic due to the destination accelerator ID being incorrect • MAC loopback controls whether traffic is looped back at the PCS layer	Global	IFoE PF	No	PROVIDER_CONFIG

Table 43: AIFM configured items

12.6 Virtual Pod Manager (VPM) Configured

Name	Parameters	FW default	Description	Scope	Config path	Config Phase
Encryption	<On Off>	Off	Whether IFoE encryption is enabled. Encryption needs to be configured consistently across the cluster.	Global	IFoE VF	TENANT_CONFIG
Encryption Key	(Index, Key)	-	Encryption key for IFoE streams.	Global or IFoE Instance?	IFoE VF	TENANT_CONFIG
Failover	<On Off>	On	Whether failover is enabled. When failover is enabled, MP_IFoE switches traffic to any alternate port on the same IFoE	Global	IFoE VF	TENANT_CONFIG

subsystem.

Table 44: VPM configured items

12.7 Firmware Configured

This configuration is handled by the MP-IFoE firmware, either based on events or derived from other values.

Name	Parameters	Description	Scope
Destination MAC/IP addresses	1024 x MAC/IP addresses	Sets the destination address (MAC or IP) for a destination accelerator ID for a specific network port. Addresses are derived from the Accelerator ID + Virtual Pod ID + Network Port ID.	Per IFoE instance, per port, per destination accelerator
Source MAC/IP address	MAC/IP address	Sets the source address (MAC or IP) for a specific network port. Addresses are derived from the Accelerator ID + Virtual Pod ID + Network Port ID.	Per IFoE instance, per port
RX Enable	Boolean	Allows the RX path to be disabled such that SDP traffic will not be sent from IFoE towards data fabric. Used to support isolation when a "Sync Flood" occurs.	Per IFoE instance
Destination checking enable	Boolean	Controls whether destination address checking is enabled. This feature only exists to support IFoE loopback i.e. it should always be enabled unless loopback is being used.	Per IFoE instance
Path to port mapping	port_num[1024][8] 2-bit port number for each of 8 paths. Mapping is per destination accelerator ID	Mapping from each of the 8 paths to the Network Port number. Separate mappings are provided for each Destination Accelerator ID to allow traffic for different streams to be directed to a specific set of Network Ports. The mapping is controlled by the MP-IFoE firmware and the configuration is dependent on the Network Port mode configured as well as Power Management and Network Port Resilience/Failover. For example, this mechanism is used to provide resilience where a Network Link becomes unreliable i.e. traffic can be redirected to avoid a specific Network Port. It is expected that in general the mapping will be the same for all destination accelerator IDs i.e. if a stream is redirected because of	Per IFoE instance, per path, per destination accelerator

		an unreliable Network Port, the same redirection would be applied to all dest IDs.	
RX Stream Configuration		Configures which stream IDs an IFoE should expect to receive. Messages with other stream IDs are dropped. This information is derived from the set of active accelerators supplied by the driver combined with the configured virtualization mode.	Per IFoE instance, per port, per stream ID
Virtual Channel Mapping Table		Table that defines whether each accelerator ID is local (within the same pod) or remote (in a different pod). Derived from local accelerator bitmask provided by AIFM.	Per IFoE instance
PIO Address Hashing	1 Port => (0, 0, 0) 2 Ports => (0xffffffff, 0, 0) 4 Ports => (0x55555555, 0xaaaaaaaa, 0)	Configures which bits of the destination address are XORed to produce the 3bit path ID for PIO traffic. Refer to IFoE Architecture mission mode for configuration to be used in production, varying according to the port mode.	Per IFoE instance, per destination accelerator
PIO Dst Accelerator Hashing	(0, 0, 0) for all port modes	Configures which bits of the destination accelerator ID are XORed to produce the 3bit path ID for PIO traffic. Refer to IFoE Architecture mission mode for configuration to be used in production.	Per IFoE instance, per destination accelerator
DMA Address Hashing	1 Port => (0, 0, 0) 2 Ports => (0xffffffff, 0, 0) 4 Ports => (0x55555555, 0xaaaaaaaa, 0)	Configures which bits of the destination address are XORed to produce the 3bit path ID for DMA traffic. Refer to IFoE Architecture mission mode for configuration to be used in production, varying according to the port mode.	Per IFoE instance, per destination accelerator
DMA Dst Accelerator Hashing	(0, 0, 0) for all port modes	Configures which bits of the destination accelerator ID are XORed to produce the 3bit path ID for DMA traffic. Refer to IFoE Architecture mission mode for configuration to be used in production.	Per IFoE instance, per destination accelerator

Table 45: Firmware configured items

12.8 Constant

These configuration items, whilst configurable in the IFoE hardware, are set to constant values by the MP_IFoE firmware for production. For certain items, firmware provides test interfaces where there is a need for specific functionality.

MP IFOE ARCHITECTURE DRAFT 1K

Parameter	HW default	FW default	Description	Test Interface
Credit Length Delta	0	0	Set credit length delta for XRSEC metadata header. Should be set to 0 - refer to mission mode.	No
Virtual Channel IDs	2 x 8-bit integer	local = 7, remote = 6	Configure which virtual channel IDs should be used for local traffic (within the same pod) and which are for remote traffic (to a different pod).	No
VLAN TPID	16-bit	0x8100	VLAN TPID field. There should be no reason to change this for different deployments, therefore constant.	No
VLAN / Priority Fields	VLAN tag, VLAN-ID, PCP/DSCP	Request Packets => (No, X., 0) Response Packets => (Yes, 0, 2)	Priority fields for Request and Response packets. Note that the HW supports different values for host and guest traffic but in mission mode the configuration is the same.	Yes

Table 46: Constant configured items

12.9 Tunables

There are many tunable parameters that may be adjusted for tuning/experimentation in production environments. These are set to the default/mission mode value currently.

Category	Parameter	HW Default	FW Default	Validation Test Config	Description
TX1 Tunables	atomic_force_en	0 (disabled)	1	0	Force atomic requests to close payloads
	atomic_rsp_force_en	0 (disabled)	TODO	0	Force atomic responses to close payloads
	pack_dma_chain	0 (disabled)	1	0	Allow dma requests to use the same path id for all parcels in a dma request

	rdreq_chain_hint_en	0 (disabled)	1	0	Allow dma read requests to hold payloads for chaining
	rdreq_force_len_thresh	0x00	3	0	Force read request payload to be closed if parcel is larger than threshold
	rdrsp_chain_hint_en	0 (disabled)	1	0	Allow read responses to hold payloads for chaining if not the last beat
	rdrsp_force_last_en	0 (disabled)	1	0	Force payload to be closed when the last beat of a read response is received
	rdrsp_force_len_thresh	0x00	3	0	Force read response payload to be closed if parcel is larger than threshold
	wrreq_chain_hint_en	0 (disabled)	1	0	Allow dma write requests to hold payloads for chaining
	wrreq_force_len_thresh	0x00	3	0	Force write request payload to be closed if parcel is larger than threshold
	wrrsp_force_last_en	0 (disabled)	1	0	Force payload to be closed when a write response is received
	wrrsp_force_len_thresh	0x00	3	0	Force write response payload to be closed if parcel is larger than threshold
TX Scheduler Tunables	max_retran	3	3	3	Maximum number of retransmissions [e.g. a value of 0 means no retransmissions]
	disable_max_retran	0 (limit enabled)	0	0	Disable maximum number of retransmission [i.e. no limit on number of retransmissions]

MP IFOE ARCHITECTURE DRAFT 1K

	enable_reason_retran	1 (enabled)	1	1	Enable IFOE_FW_REASON_RETRAN messages on firmware response interface
	enable_reason_wake	0 (disabled)	0	0	Enable IFOE_FW_REASON_WAKE messages on firmware response interface
	enable_reason_active	0 (disabled)	0	0	Enable IFOE_FW_REASON_ACTIVE messages on firmware response interface
	enable_boost_req	1 (enabled)	1	1	Enable high priority boost for Write Engine requests
	enable_boost_res	1 (enabled)	1	1	Enable high priority boost for Write Engine responses
	enable_boost_ack	1 (enabled)	1	1	Enable high priority boost for pure ACKs
	enable_boost_retran	1 (enabled)	1	1	Enable high priority boost for retransmissions
	enable_port_map	0 (disabled)	0	0	Enable path+gpu to port mapping
TX Scheduler Priority Channel Tunables	net_credit_test	0 (normal operation, no testing)	0	0	Enable network credit test modes
	con_credit_test	0 (normal operation, no testing)	0	0	Enable congestion credit test modes
	con_credit_chan	0 (don't use request/response bit)	0	0	Control use of request/response bit of priority channel in ifoe_tx_sched_pchan congestion credit control. This is related to DEIFOEPM-60 - Add window flow control on per-class-per-port basis Closed

	con_credit_enable	0 (disabled)	TODO	???	Enable congestion credit control
	net_credit_enable	0 (disabled)	TODO	???	Enable network credit control
	ddr_enable	1 (enabled)	1	1	Enable ifoe_tx_sched_pchan deficit-round-robin (DRR) processing
	net_credits	1280 (on all channels)	TODO	1280	Network credit offset on each priority channel [in units of 32B]. There are a maximum of 16 channels (2 per port).
	net_overhead	$40 + 4 * (i \% 2)$ for channel ID i (0..15)	TODO	TODO	Number of overhead bytes in packets on each priority channel [in bytes] There are a maximum of 16 channels (2 per port)
	net_qidx	i for channel ID i (0..15)	TODO	TODO	Network queue index for each priority channel There are a maximum of 16 channels (2 per port)
	con_credits	43750 (on all channels)	TODO	43750	Congestion credit offset on each priority channel [in units of 8B] There are a maximum of 16 channels (2 per port)
TX Scheduler RX Ack Tunables	open_loop	0 (open loop disabled)	0	0	Enable open_loop mode [packets will be automatically ACKd on first transmission]
	timer_backoff	1 (enabled)	1	1	Determine if rxack_timer has exponential backoff on subsequent retransmissions
	timer_disable	0 (timer enabled)	0	0	Disable rxack_timer [packets are never automatically NAKd]
	timer	25 (microseconds)	25	25	Number of timestamp ticks before a packet is automatically NAKd
	timestamp_div	1850 (1us @1.85GHz)	1850	1850	Value to divide IFoE clock to obtain local timestamp clock [a value of 0 is

MP IFOE ARCHITECTURE DRAFT 1K

					divide-by-65536]
TX Scheduler TX Ack Tunables	timer_disable	0 (timer enabled)	0	0	Disable txack_timer [packets are never automatically NAKd]
	timer	25 (microseconds)	500 (ns)	25	Number of timestamp ticks before a packet is automatically NAKd
	timestamp_div	1850 (1us @1.85GHz)	1850	1850	Value to divide IFOE clock to obtain local timestamp clock [a value of 0 is divide-by-65536]
SDP RX Meta-Data	request_network_id	0x0	0		value to use for the network id when returning request credits
	response_network_id	0x1	1		value to use for the network id when returning response credits
	stream_id	0	0		value to use for M_SDP field req_stream_id bits 15:1 as these are to be driven to a constant value
	pass_pw	0	0		value to use for M_SDP field req_pass_pw as this is to be driven to a constant value
	io	0	0		value to use for M_SDP field req_io as this is to be driven to a constant value
	at	0	0		value to use for M_SDP field req_at as this is to be driven to a constant value
	pasid	0	0		value to use for M_SDP field req_pasid as this is to be driven to a constant value
	pasidvalid	0	0		value to use for M_SDP field req_pasid_valid as this is to be driven to a constant value

TX Buffer Tunables	mtu_threshold_req	0x80	128 = 4096 bytes	Maximum length of request IFOE payloads generated by the scheduler before it splits a payload into multiple packets. Actual maximum value is higher
	mtu_threshold_res	0x80	128 = 4096 bytes	Maximum length of response IFOE payloads generated by the scheduler before it splits a payload into multiple packets. Actual maximum value is higher
	tx2_ack_timeout_threshold_ns	Not in HW	TODO	Max range of 32us, expected implementation has a resolution of 10ns
	timer_theshold	0x10	0	Timer threshold (16/1.8G granularity approx. 9ns)
	timer_priority_req	0 (sched_ack)	0	Wake operations only generated when timer expires for request streams. enum sched_ack: 0, timer: 1
	timer_priority_res	0 (sched_ack)	0	Wake operations only generated when timer expires for response streams. enum sched_ack: 0, timer: 1
	pkt_force_priority_req	1 (high)	TODO	Priority low/high of forced packets for request streams
	pkt_hold_enable_req	1 (enabled)	1	Enables packet hold pending scheduler acknowledgements for request streams
	pkt_force_priority_res	1 (high)	TODO	Priority low/high of forced packets for response streams
	pkt_hold_enable_res	1 (enabled)	1	Enables packet hold pending scheduler acknowledgements for response streams

Table 47: Tunable configured items

13 Appendix D: CReST testing

As part of silicon validation for a MID, CReST testing needs to be performed to check that the MID is functioning correctly before it is packaged with the other MI450 components. The testing should check as much of the functionality as is possible and be as fast as practicable. This functionality is likely to be in a special firmware image but the functionality is described here for completeness.

The IFoE system has complicated programming sequences to bring the device into operation as each subsystem contains several IP blocks (IFoE, XRSEC, XRPFC, XRMAC). Additionally, the MP_IFoE has to interact with the (GTA) PHYs associated each subsystem. These PHYs have their own firmware to manage them.

The proposed testing is to put the GTA PHYs in loopback at the lowest possible level (so that as much IP is validated as possible), configure the IFoE components and use PIE to send transactions down each IFoE subsystem through the DXS and receive them back from the DXM for verification.

13.1 Workflow

The number of IFoE systems, the amount of (register) configuration and interactions with the IP blocks, and the need to use the GTA PHY means that using just register writes to configure a system is infeasible. In the following, TC is the test controller which writes to and reads from the MID.

The proposed workflow considering only the IFoE related element (excluding any data fabric programming would be as follows):

- TC configures KPX for GTA PHY operation
- TC loads GTA firmware onto each GTA PHY (believed to be per quad or per lane)
- TC performs GTA startup operations
- TC performs other configuration necessary for MP_IFoE operation (for example, programming RSMU)
- TC loads MP_IFoE CReST-specific firmware image into each MP_IFoE's SRAM on the MID
- TC releases MP_IFoE reset, allowing the MP_IFoE to run and perform basic configuration

At this point, the MID is "ready for test". The MP_IFoE firmware provides simple mailbox-like operations to control the tests. The TC writes a command to a mailbox and reads back from (say) a known memory location. The operations available would include:

- MP_IFoE is ready (the MP_IFoE actually ran)
- Set IFoE sub-system #X for loopback. Configures PHY and IFoE for loopback
- Enable security block (MP_IFoE). XRSEC can be selectively enabled so testing with and without would be beneficial
- Run transaction test. Allows MP_IFoE to note any detail relevant to test
- Read transaction results. Reports any errors observed during testing

TODO: Mailboxes and description

14 Appendix E: Telemetry identifiers

The telemetry datasets identify each telemetry element with a unique identifier. This allows external systems to extract the data correctly.

TODO: Add when available